LARGE LANGUAGE MODELS FOR OPTIMIZATION USING PROMPTING AND REASONING

A Project Report

submitted by

PARAS MUKATI

in partial fulfilment of the requirements

for the award of the degree of

MASTER OF TECHNOLOGY

DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS.

MAY 2026

THESIS CERTIFICATE

This is to certify that the thesis titled **LARGE LANGUAGE MODELS FOR OPTIMIZATION USING PROMPTING AND REASONING**, submitted by **Paras Mukati** to the Indian Institute of Technology Madras, for the award of the degree of **Dual Degree (B.Tech + M.Tech)**, is a bona fide record of research work carried out by him under our supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Prof. Dr. Sivaram Ambikasaran
Associate Professor
Dept. of Data Science and AI
IIT Madras

Dr. Sri Vallabha Deevi
Adjunct Faculty
Dept. of Data Science and AI
IIT Madras

Place: IIT Madras, Chennai, Tamil Nadu - 600036

Date: May 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Prof. Dr. Sivaram Ambikasaran**, for his invaluable guidance, support, and encouragement throughout the course of this work. His insights and feedback have been instrumental in shaping this research.

I also extend my thanks to **Dr. Sri Vallabha Deevi** for his guidance and constructive suggestions during this work.

I am grateful to the faculty and staff of the Department of Data Science and Artificial Intelligence at IIT Madras for providing a stimulating academic environment.

I express my heartfelt gratitude to my family for their unwavering support.

ABSTRACT

Optimization is a fundamental problem in operations research and computer science, particularly in domains involving combinatorial decision-making such as scheduling, routing, and resource allocation. Traditional optimization techniques often require explicit mathematical formulations, which limit their flexibility.

Recent advancements in large language models (LLMs) have opened new avenues for solving optimization problems using natural language. One such approach, Optimization by Prompting (OPRO), utilizes LLMs as iterative optimizers.

In this work, the effectiveness of the OPRO framework is evaluated across a range of classical optimization problems. Results indicate that while OPRO performs well on simpler problems, its performance is inconsistent on more complex tasks. This study motivates the need for incorporating structured reasoning (ReAct) to improve solution quality.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS **1**

ABSTRACT **2**

1 INTRODUCTION **14**

1.1 Introduction to Optimization 14

1.2 Traditional Optimization Approaches 14

1.3 Large Language Models as Optimizers 15

1.4 OPRO Framework 16

1.4.1 Meta-Prompt Design 16

1.4.2 Optimization Loop 17

1.4.3 Exploration vs Exploitation 17

1.5 Implementation of OPRO 17

1.6 Benchmark Problems 17

1.6.1 Classic 0/1 Knapsack Problem 17

1.6.2 Multi-constraint / Multi-dimensional Knapsack (MDKP) . . 18

1.6.3 Bounded Knapsack Problem 18

1.6.4 Travelling Salesman Problem (TSP) 19

1.6.5 Vehicle Routing Problem (VRP) 19

1.6.6 Job-Shop Scheduling Problem 20

1.6.7 Bin Packing Problem 20

1.6.8 Assignment Problem 20
1.6.9 Maximum Flow Problem 21
1.6.10 Minimum Cost Flow Problem 22
1.7 Experimental Results 22
1.7.1 Model-wise Performance 22
1.7.2 Observations 23
1.8 Limitations of OPRO 23
1.9 Chapter Summary 23

2 OPRO + ReAct Framework **24**

2.1 Introduction 24
2.2 Motivation 24
2.3 Proposed Framework: OPRO + ReAct 26
2.4 Algorithm Description 27
2.5 Key Components of the Framework 29
2.5.1 LLM-based Code Generator 29
2.5.2 Execution and Feasibility Evaluator 29
2.5.3 ReAct-based Critic Module 30
2.5.4 Sequence Reconstruction using SSG 30
2.5.5 Solution History and Selection Mechanism 30
2.5.6 Baseline Integration 31
2.6 Experimental Setup 31
2.6.1 Model Configuration 31
2.6.2 Problem Instances and Initialization 31
2.6.3 Solution Generation and Execution 31

2.6.4 Evaluation Criteria 32
2.6.5 Iterative Optimization Process 32
2.6.6 Solution History and Pruning 32
2.6.7 Termination Criteria 33
2.6.8 Performance Metrics 33
2.7 Results 33
2.7.1 Classic 0/1 Knapsack Problem 34
2.7.2 Multi-constraint Knapsack Problem 36
2.7.3 Bounded Knapsack Problem 37
2.7.4 Multi-Dimensional Knapsack Problem 39
2.7.5 Multi-Objective Knapsack Problem 42
2.7.6 Travelling Salesman Problem (TSP) 45
2.7.7 Vehicle Routing Problem (VRP) 47
2.7.8 Job-Shop Scheduling Problem (JSSP) 49
2.7.9 Bin Packing Problem 52
2.7.10 Assignment Problem 53
2.7.11 Generalized Assignment Problem (GAP) 55
2.7.12 Maximum Flow and Minimum Cut 57
2.7.13 Minimum-Cost Flow 59
2.8 Comparative Analysis 60
2.9 Discussion 61
2.10 Limitations and Open Questions 62
2.11 Chapter Summary 63
2.12 Model-wise Observations 63

3 Hospital Scheduling Problem **65**

3.1 Introduction 65

3.2 Problem Formulation 66

3.3 Mathematical Model 67

3.4 Baseline Optimization Approaches 68

3.4.1 Column Generation Approach 68

3.4.2 Cutting Plane Approach 70

3.4.3 Benders Decomposition Approach 71

3.5 LLM-Augmented Optimization Framework 72

3.5.1 LLM-Augmented Column Generation 73

3.5.2 LLM-Augmented Cutting Plane Method 74

3.5.3 LLM-Augmented Benders Decomposition 75

3.5.4 Role of ReAct and Feedback Loop 76

3.6 Experimental Setup 77

3.6.1 Problem Instance Generation 77

3.6.2 Baseline Methods 77

3.6.3 OPRO + ReAct Implementation 78

3.6.4 Model Selection 78

3.6.5 Evaluation Metrics 79

3.6.6 Experimental Protocol 79

3.6.7 Comparison Strategy 79

3.7 Results 80

3.7.1 Column Generation Results 80

3.7.2 Cutting Plane Results 84

3.7.3 Benders Decomposition Results 87
3.8 Comparative Evaluation 89
3.9 Discussion 90
3.10 Chapter Summary 91

4 Flight Crew Assignment **92**
4.1 Introduction 92
4.2 Problem Formulation 92
4.3 Mathematical Formulation 93
4.4 Optimization Frameworks 94
4.4.1 Column Generation Framework 95
4.4.2 Cutting Plane Framework 96
4.4.3 Benders Decomposition Framework 97
4.4.4 Role of OPRO + ReAct 98
4.5 Results 98
4.5.1 Column Generation Performance 99
4.5.2 Comparison with Baseline 101
4.5.3 Convergence Behavior 101
4.5.4 Scalability 102
4.5.5 Summary 102
4.5.6 Cutting Plane Results 102
4.5.7 Benders Decomposition Results 105
4.6 Comparative Evaluation 108
4.7 Discussion 109
4.8 Chapter Summary 109

5 Conclusion and Future Work **111**

5.1 Conclusion 111

5.2 Future Work 112

5.3 Final Remarks 112

LIST OF TABLES

1.1 Model-wise Performance Comparison across Combinatorial Problems 22

2.1 Node coordinates and demand for VRP instance (Node 0 is the depot) 47

2.2 Job-Shop Scheduling instance showing operation sequence, machine assignment, and processing time 50

2.3 GAP assignment comparison between Greedy and ReAct+OPRO approaches 56

3.1 Baseline Column Generation performance 80

3.2 OPRO + ReAct Column Generation performance 81

3.3 Baseline Cutting Plane performance 84

3.4 OPRO + ReAct Cutting Plane performance 85

3.5 Baseline Benders Decomposition performance 87

3.6 OPRO + ReAct Benders Decomposition performance 88

4.1 Comparison of baseline and proposed methods 101

4.2 Baseline Cutting Plane performance 102

4.3 OPRO + ReAct Cutting Plane performance 103

4.4 Baseline Benders performance 105

4.5 OPRO + ReAct Benders performance 106

LIST OF FIGURES

1.1 Enhanced Optimization by Prompting (OPRO) framework. The LLM iteratively generates candidate solutions in the form of executable code, evaluates them using an objective function, and updates the meta-prompt using solution-score pairs. Upon completion, the top-performing solutions are returned. 16

1.2 Graph representation of the Maximum Flow instance 21

2.1 Refined OPRO + ReAct framework. The system evaluates generated solutions against a baseline and iteratively refines them using reasoning-based feedback until optimality is achieved. 26

2.2 LLaMA 3.1 8B performance on 0/1 Knapsack. 34

2.3 LLaMA 3.3 70B performance on 0/1 Knapsack. 35

2.4 GPT-OSS 120B performance on 0/1 Knapsack. 35

2.5 LLaMA 3.1 8B performance on MCKP. 36

2.6 LLaMA 3.3 70B performance on MCKP. 37

2.7 GPT-OSS 120B performance on MCKP. 37

2.8 GPT-OSS 120B performance on bounded knapsack. 38

2.9 LLaMA 3.1 8B performance on bounded knapsack. 38

2.10 LLaMA 3.3 70B performance on bounded knapsack. 39

2.11 LLaMA 3.1 8B performance on multi-dimensional knapsack. 41

2.12 LLaMA 3.3 70B performance on multi-dimensional knapsack. . . . 41

2.13 GPT-OSS 120B performance on multi-dimensional knapsack. . . . 41

2.14 LLaMA 3.1 8B Pareto solutions (3 iterations). 43

2.15 LLaMA 3.3 70B Pareto solutions (3 iterations). 43

2.16 GPT-OSS 120B Pareto solutions (5 iterations). 44

2.17 TSP solution using LLaMA 8B (3 iterations). 45

2.18 TSP solution using LLaMA 70B (2 iterations). 46

2.19 TSP solution using GPT-OSS 120B (2 iterations). 46

2.20 VRP solution using LLaMA 8B (3 iterations). 48

2.21 VRP solution using LLaMA 70B (3 iterations). 48

2.22 VRP solution using GPT-OSS 120B (3 iterations). 48

2.23 JSSP schedule using LLaMA 8B (8 iterations). 50

2.24 JSSP schedule using LLaMA 70B (6 iterations). 50

2.25 JSSP schedule using GPT-OSS 120B (3 iterations). 51

2.26 Bin packing using LLaMA 8B (6 iterations). 52

2.27 Bin packing using LLaMA 70B (6 iterations). 52

2.28 Bin packing using GPT-OSS 120B (6 iterations). 52

2.29 Assignment Problem: Optimal (Hungarian) vs ReAct+OPRO LLM solution 54

2.30 Maximum Flow Comparison 58

2.31 Graph with Minimum Cut Highlighted 58

2.32 Flow Comparison 59

2.33 Min-Cost Flow Visualization 60

3.1 Column Generation framework for hospital scheduling. The algorithm iteratively generates feasible schedules (columns) using a pricing subproblem until no improving column is found. 70

3.2 Cutting Plane method. The algorithm iteratively detects violated constraints and refines the feasible region until all constraints are satisfied. 71

3.3 Benders Decomposition framework. The master problem proposes assignments, and the subproblem verifies feasibility, generating cuts when necessary. 72

3.4 LLM-augmented Column Generation. The LLM proposes candidate columns which are validated using exact pricing. 74

3.5 LLM-assisted Cutting Plane method. The LLM identifies candidate cuts which are validated before inclusion. 75

3.6 LLM-assisted Benders Decomposition. The LLM identifies infeasibilities which are converted into cuts. 76

3.7 Baseline Column Generation: convergence and final schedule (24 patients) 81

3.8 OPRO + ReAct Column Generation for 24 patients 82

3.9 OPRO + ReAct Column Generation for 50 patients 82

3.10 OPRO + ReAct Column Generation for 100 patients 83

3.11 Baseline Cutting Plane schedule (24 patients) 84

3.12 OPRO + ReAct Cutting Plane schedule (24 patients) 85

3.13 OPRO + ReAct Cutting Plane schedule (50 patients) 86

3.14 OPRO + ReAct Cutting Plane schedule (100 patients) 86

3.15 Baseline Benders Decomposition: convergence and final schedule (18 patients) 87

3.16 OPRO + ReAct Benders Decomposition: convergence and final schedule (18 patients) 88

4.1 Column Generation with LLM-guided iterative pricing 96

4.2 Cutting Plane with iterative cut generation and LLM guidance . . . 97

4.3 Benders Decomposition with iterative feasibility feedback 98

4.4 Baseline Column Generation: Crew routing for 24 flights 99

4.5 Proposed Method: Convergence and routing for 24 flights 100

4.6 Proposed Method: Convergence and routing for 50 flights 100

4.7 Proposed Method: Convergence and routing for 100 flights 101

4.8 Baseline Cutting Plane schedule (22 flights) 103

4.9 OPRO + ReAct Cutting Plane schedule (22 flights) 103

4.10 OPRO + ReAct Cutting Plane schedule (50 flights) 104

4.11 OPRO + ReAct Cutting Plane schedule (100 flights) 104

4.12 Baseline Benders schedule (12 flights) 106

4.13 OPRO + ReAct Benders schedule (22 flights) 107

CHAPTER 1

INTRODUCTION

1.1 Introduction to Optimization

Optimization refers to the process of finding the best possible solution from a set of feasible alternatives, subject to given constraints. It plays a central role in a wide range of applications including scheduling, logistics, resource allocation, finance, and engineering design.

Optimization problems can broadly be categorized into two types: continuous and combinatorial. Continuous optimization (Chambolle 2022) deals with variables that take real values and is often solved using gradient-based methods. In contrast, combinatorial optimization (Nemhauser and Williams 1990) involves discrete decision variables, where the solution space is finite but grows exponentially with problem size.

Combinatorial optimization problems, such as the knapsack problem, traveling salesman problem, and scheduling tasks, are particularly challenging due to their inherent computational complexity. As the number of variables increases, the number of possible solutions increases exponentially, making exhaustive search infeasible. Many of these problems are NP-hard, requiring advanced techniques or approximations to obtain near-optimal solutions within reasonable time.

1.2 Traditional Optimization Approaches

Traditional approaches to optimization can be broadly classified into exact methods and heuristic methods.

Exact methods, such as Linear Programming (LP) and Integer Programming (IP), aim to find provably optimal solutions. These methods rely on mathematical formulations and are typically solved using techniques such as branch-and-bound (Land and

Doig 1960), cutting planes (Bùi and Marchi 2025), and dynamic programming (Bellman 1957). While they guarantee optimality, their computational cost becomes prohibitive for large-scale combinatorial problems.

Heuristic and metaheuristic methods, including greedy algorithms (Yao, Hao, and Zhang 2023) and local search techniques, are often used to obtain approximate solutions efficiently. These methods trade optimality for computational speed and are widely used in practical applications.

Modern optimization solvers, such as OR-Tools and CP-SAT, provide powerful frameworks for solving structured optimization problems. However, these approaches require explicit problem modeling, careful constraint formulation, and domain expertise.

Overall, traditional optimization methods are highly effective but suffer from limited flexibility and scalability, especially when dealing with diverse problem formulations. These methods require problem-specific modeling and often struggle to generalize across different domains.

1.3 Large Language Models as Optimizers

Recent advancements in large language models (LLMs) have introduced a new paradigm for solving optimization problems. LLMs possess strong capabilities in natural language understanding, pattern recognition, and reasoning, enabling them to interpret problem descriptions and generate structured outputs.

Unlike traditional optimization approaches that rely on explicit mathematical formulations, LLM-based methods treat optimization as a generative process. A problem can be described in natural language, and the model generates candidate solutions based on learned patterns and prior context.

This perspective allows optimization to be framed as a sequence generation task, where the model iteratively improves solutions based on feedback. Such an approach reduces the need for manual modeling and enables a more flexible framework applicable across different problem domains.

One such framework is Optimization by Prompting (OPRO) (Yang et al. 2023), where the LLM acts as an optimizer that refines solutions through iterative prompting and evaluation.

1.4 OPRO Framework

The Optimization by Prompting (OPRO) framework utilizes large language models to iteratively generate candidate solutions in the form of executable code, which are then used to compute optimal or near-optimal solutions. Instead of directly outputting final solutions, the LLM produces code that encodes the optimization logic. These generated programs are evaluated using an objective function, and their outcomes are fed back into the prompt. In this way, the optimization process is guided through prompts that incorporate historical solutions and their corresponding objective values.

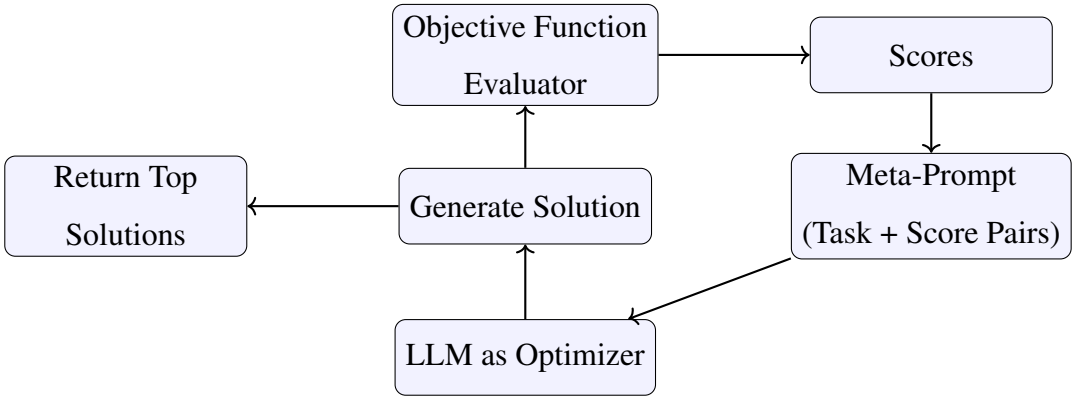

Figure 1.1: Enhanced Optimization by Prompting (OPRO) framework. The LLM iteratively generates candidate solutions in the form of executable code, evaluates them using an objective function, and updates the meta-prompt using solution-score pairs. Upon completion, the top-performing solutions are returned.

1.4.1 Meta-Prompt Design

The meta-prompt is a central component of the OPRO framework. It consists of the task description along with a set of previously generated solutions and their associated objective scores. This structure allows the LLM to learn from past performance and generate improved solutions.

Each iteration augments the meta-prompt with new solution-score pairs, forming an

optimization trajectory that guides future generations.

1.4.2 Optimization Loop

The OPRO framework follows an iterative loop:

1. Generate candidate solutions using the LLM
2. Evaluate each solution using the objective function
3. Append the solution and its score to the meta-prompt
4. Repeat the process until convergence or stopping criteria are met

1.4.3 Exploration vs Exploitation

To balance exploration and exploitation, OPRO leverages techniques such as temperature control and multiple solution generations.

1.5 Implementation of OPRO

In this work, the OPRO framework is implemented using multiple large language model APIs, including LLaMA 3.1 8B, LLaMA 3.3 70B, and GPT-OSS 120B.

1.6 Benchmark Problems

The performance of the OPRO framework is evaluated on a diverse set of classical combinatorial optimization problems. Each problem is defined with specific numerical instances to ensure consistent evaluation across models.

1.6.1 Classic 0/1 Knapsack Problem

The knapsack problem models decision-making under limited resources.

Example: Selecting items for a backpack where weight capacity is limited.

Given items with values and weights, the objective is to maximize total value without exceeding capacity.

Instance:

- Values: [60, 100, 120]
- Weights: [10, 20, 30]
- Capacity: 50

Explanation: Each item has a value (benefit) and weight (cost). The goal is to choose a subset of items such that the total weight is within capacity while maximizing total value.

1.6.2 Multi-constraint / Multi-dimensional Knapsack (MDKP)

This problem extends the knapsack by introducing multiple constraints.

Example: Packing cargo where both weight and volume limits must be satisfied.

Instance:

- Values: [100, 200, 150]
- Weight Constraint: [20, 30, 40], Limit = 60
- Volume Constraint: [10, 10, 10], Limit = 25

Explanation: Each item consumes multiple resources (e.g., weight and volume). A feasible solution must satisfy all constraints simultaneously.

1.6.3 Bounded Knapsack Problem

Each item is available in limited quantity.

Example: Selecting products from inventory where stock is limited.

Instance:

- Values: [10, 40, 30, 50]
- Weights: [5, 4, 6, 3]

- Quantities: [3, 1, 2, 2]
- Capacity: 20

Explanation: Unlike 0/1 knapsack, items can be chosen multiple times, but not exceeding their available quantity.

1.6.4 Travelling Salesman Problem (TSP)

The objective is to find the shortest route that visits each city exactly once and returns to the starting city.

Example: A delivery agent planning the shortest route covering all cities.

Distance Matrix Representation:

	C_0	C_1	C_2	C_3
C_0	0	10	15	20
C_1	10	0	35	25
C_2	15	35	0	30
C_3	20	25	30	0

Explanation: Rows and columns represent cities. The entry at (i, j) denotes the distance from city i to city j. For example, the distance from C_0 to C_2 is 15.

1.6.5 Vehicle Routing Problem (VRP)

An extension of TSP where multiple vehicles serve customers.

Example: Delivery trucks distributing packages across locations.

Instance:

- Number of Vehicles: 3
- Vehicle Capacity: 15
- Customer Demands: [0, 5, 8, 3, 6, 4]

Explanation: Each vehicle must serve a subset of customers such that total demand on each route does not exceed vehicle capacity.

1.6.6 Job-Shop Scheduling Problem

Jobs must be processed on machines in a predefined order.

Example: Manufacturing products on different machines in sequence.

Instance:

- Job 0: Machine A (3), Machine B (2)
- Job 1: Machine B (2), Machine A (4)

Explanation: Each job consists of ordered operations. For example, Job 0 must first be processed on Machine A for 3 time units, followed by Machine B for 2 time units. Machines can process only one job at a time.

1.6.7 Bin Packing Problem

The objective is to pack items into the minimum number of bins.

Example: Packing goods into boxes with limited capacity.

Instance:

- Item Weights: [4, 8, 1, 4, 2, 1]
- Bin Capacity: 10

Explanation: Items must be grouped into bins such that the sum of weights in each bin does not exceed capacity.

1.6.8 Assignment Problem

The goal is to assign tasks to workers at minimum cost.

Example: Assigning employees to projects to minimize total cost.

Cost Matrix Representation:

	T_1	T_2	T_3	T_4
W_1	9	2	7	8
W_2	6	4	3	7
W_3	5	8	1	8
W_4	7	6	9	4

Explanation: Rows represent workers (W_i) and columns represent tasks (T_j). Each entry gives the cost of assigning worker i to task j.

1.6.9 Maximum Flow Problem

The objective is to maximize flow from a source to a sink.

Example: Maximizing throughput in a transportation or communication network.

Instance:

- Source $\rightarrow$ A: 16
- Source $\rightarrow$ B: 13
- A $\rightarrow$ C: 12
- B $\rightarrow$ A: 4
- C $\rightarrow$ Sink: 20

Graph Representation:

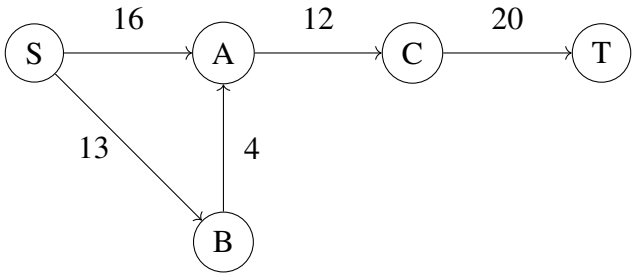

Figure 1.2: Graph representation of the Maximum Flow instance

Explanation: Edges represent connections with capacities. Flow must respect capacity limits while maximizing total flow from source to sink.

1.6.10 Minimum Cost Flow Problem

The goal is to send a required amount of flow at minimum cost.

Example: Transporting goods through a network at the lowest cost.

Instance:

- Required Flow: 20 units
- Example Edge: Capacity = 10, Cost per unit = 2

Explanation: Each edge has a capacity and cost. The objective is to route the required flow while minimizing total transportation cost.

1.7 Experimental Results

1.7.1 Model-wise Performance

Table 1.1: Model-wise Performance Comparison across Combinatorial Problems

Problem	LLaMA 3.1 8B	LLaMA 3.3 70B	GPT-OSS 120B
0/1 Knapsack	✓	✓	✓
Multi-constraint Knapsack	✓	✓	✓
Bounded Knapsack	✗	✓	✓
MDKP	✗	✓	✓
Multi-objective Knapsack	✗	✓	✓
TSP	✗	✓	✓
VRP	✗	✗	✗
Job Scheduling	✗	✗	✓
Assignment Problem	✗	✓	✓
Max Flow / Min Cut	✗	✗	✓
Min Cost Flow	✗	✗	✓

✓ indicates successful optimal solution; ✗ indicates failure or infeasible output.

1.7.2 Observations

The experimental results reveal clear trends in the performance of different language models across optimization problems. Smaller models tend to struggle with complex and highly structured problems, often failing to generate feasible or optimal solutions. In contrast, larger models demonstrate significantly better performance due to their enhanced reasoning and pattern recognition capabilities. However, certain classes of problems, particularly vehicle routing and network flow problems, remain challenging even for larger models. These problems involve tightly coupled constraints and structured decision spaces, which are difficult for purely generative approaches to handle effectively.

1.8 Limitations of OPRO

Despite its flexibility and general applicability, the OPRO framework exhibits several important limitations. The optimization process is often unstable, with solution quality varying significantly across iterations. Additionally, the framework lacks consistent convergence behavior, making it difficult to guarantee improvement over time. The performance is highly sensitive to prompt design and initialization, which introduces variability in results. Furthermore, OPRO struggles to handle problems with strict combinatorial constraints, where feasibility plays a critical role. These limitations highlight the need for incorporating structured reasoning and feedback mechanisms, which motivates the approach presented in the next chapter.

1.9 Chapter Summary

This chapter introduced optimization and OPRO. Results show that OPRO works for simpler problems but struggles with complex ones, motivating improvements in the next chapter.

CHAPTER 2

OPRO + ReAct Framework

2.1 Introduction

Chapter 1 demonstrated that while the Optimization by Prompting (OPRO) framework provides a flexible approach for solving optimization problems using large language models, it suffers from several limitations. In particular, the performance of OPRO is inconsistent across problem types and highly dependent on model size and prompt design.

A key limitation of OPRO is the absence of structured reasoning and feedback. The model generates solutions in an iterative manner, but lacks the ability to explicitly analyze errors or refine solutions based on past failures.

To address these limitations, this chapter proposes an enhanced framework that integrates reasoning and iterative feedback into the OPRO process, inspired by the ReAct paradigm.

2.2 Motivation

The primary motivation behind this work is to improve the stability and reliability of LLM-based optimization.

In the standard OPRO framework, the model generates candidate solutions without explicit reasoning or structured analysis. As a result, there is no systematic mechanism to evaluate incorrect or suboptimal outputs, which limits the ability of the model to learn from its mistakes. Consequently, the improvement in solution quality across iterations is often inconsistent, leading to unstable optimization performance.

In contrast, human problem-solving involves reasoning about mistakes and iteratively refining solutions. This suggests that incorporating a reasoning loop within the optimization process could significantly improve performance.

Therefore, we propose integrating a ReAct-style feedback mechanism, where the model:

- Evaluates its own outputs
- Identifies areas for improvement
- Updates future generations based on structured feedback

While this framework demonstrates strong performance on benchmark problems, an important question remains:

Can this approach scale to tightly constrained and complex real-world optimization problems?

This question is explored in Chapters 3 and 4.

2.3 Proposed Framework: OPRO + ReAct

The proposed framework extends OPRO by introducing a reasoning-based feedback loop.

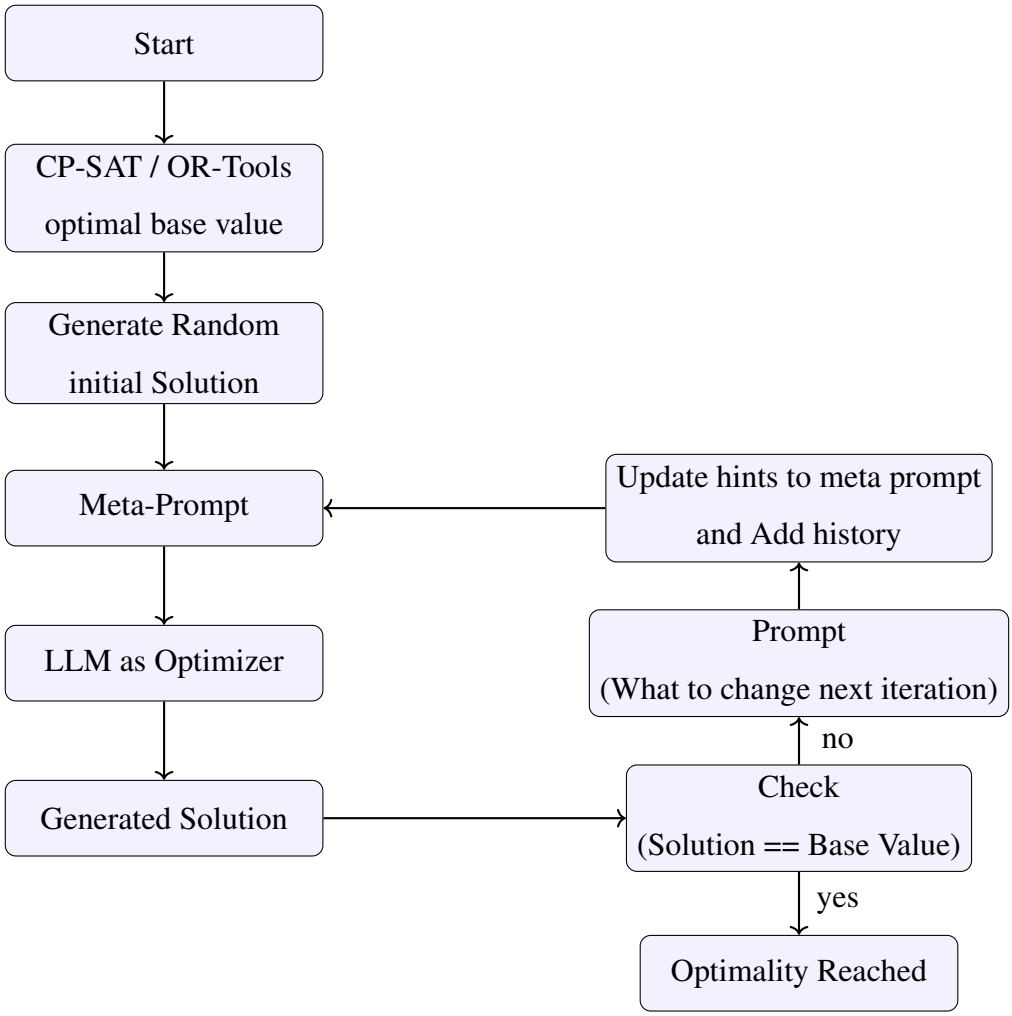

Figure 2.1: Refined OPRO + ReAct framework. The system evaluates generated solutions against a baseline and iteratively refines them using reasoning-based feedback until optimality is achieved.

The workflow consists of the following steps:

1. Initialize using a baseline solution (e.g., OR-Tools or CP-SAT)
2. Generate an initial candidate solution
3. Construct a meta-prompt incorporating previous solutions
4. Use the LLM to generate improved solutions
5. Evaluate the generated solution
6. If not optimal:

- Ask the model for improvement hints
- Update the meta-prompt with feedback

7. Repeat until convergence

2.4 Algorithm Description

The proposed OPRO + ReAct framework follows a hybrid optimization strategy that combines LLM-based program generation, execution-driven evaluation, and iterative reasoning-based refinement.

Unlike standard OPRO approaches that directly generate candidate solutions, the framework employed in this work generates executable code that constructs feasible schedules algorithmically. These generated programs are executed in a controlled environment, and the resulting schedules are evaluated based on feasibility and objective value.

The overall procedure begins with the generation of a baseline solution using a classical optimization solver (OR-Tools CP-SAT). This baseline serves as a reference for evaluating improvements in subsequent iterations.

At each iteration, the framework operates in two complementary phases:

1. LLM-based Candidate Generation: A structured prompt is constructed using the problem definition and a subset of previously successful solutions. The LLM generates Python code that attempts to construct a feasible schedule by exploring operation orderings while respecting precedence and machine capacity constraints. The generated code is executed, and the resulting schedule is validated. Only feasible solutions with valid objective values are retained.

2. ReAct-based Critic Refinement: In addition to code generation, a reasoning-based critic module analyzes the current best solution. Instead of regenerating solutions from scratch, the critic proposes localized modifications such as swapping operations, relocating tasks, or reversing sequences on specific machines. These edits are applied to machine-level sequences, and a Serial Insertion Schedule Generation procedure (Artigues, Michelon, and Reusser 2003)is used to reconstruct a feasible schedule. The modified solution is accepted only if it improves the objective value.

The interaction between these two phases enables both global exploration (via LLM-generated programs) and local exploitation (via critic-driven refinements), resulting in a more stable and efficient optimization process.

The algorithm also maintains a history of candidate solutions, which is periodically pruned to retain only the best-performing schedules. This ensures that the search remains focused on high-quality regions of the solution space.

The overall procedure can be summarized as follows:

```
Initialize baseline solution using OR-Tools
Store baseline in history

for each iteration:

    # ReAct Critic Phase
    Select best solution from history
    Generate critic feedback (swap/move/reverse operations)
    Apply edits to machine sequences
    Reconstruct schedule using SSG
    If improved:
        Add to history

    # LLM Generation Phase
    Build prompt using problem + history
    Generate candidate code using LLM
    Execute generated code
    Extract schedule and evaluate feasibility
    If valid:
        Add to history

    # Prune history
    Keep top-k solutions based on objective value

Return best solution found
```

This formulation closely aligns with the implemented system for Job-Shop Scheduling, where feasibility is strictly enforced through precedence and machine constraints, and improvements are guided through both generative and reasoning-based mechanisms.

2.5 Key Components of the Framework

The proposed OPRO + ReAct framework is composed of several interconnected components that collectively enable iterative optimization through generation, evaluation, and refinement. These components are tightly integrated to ensure that candidate solutions are both feasible and progressively improved across iterations.

2.5.1 LLM-based Code Generator

The primary component of the framework is the LLM-based code generator, which produces executable programs that construct candidate solutions. Unlike traditional optimization approaches that directly output solutions, the LLM generates Python code that encodes the logic required to explore feasible schedules. The prompt provided to the model includes the problem definition, constraints, and selected high-quality solutions from previous iterations. This enables the model to generate structured algorithms that explore operation orderings while respecting precedence and machine capacity constraints. As a result, the generation process is guided and informed rather than purely random.

2.5.2 Execution and Feasibility Evaluator

Once a candidate program is generated, it is executed in a controlled environment to produce a schedule. The resulting schedule is then evaluated for both feasibility and objective value. Feasibility is ensured by verifying that precedence constraints between operations are satisfied, that no machine processes more than one operation at a time, and that all required operations are scheduled. If any of these conditions are violated, the candidate is rejected. Valid solutions are assigned an objective value, such as makespan, which is used to compare and rank candidate schedules.

2.5.3 ReAct-based Critic Module

To enhance solution quality, a reasoning-based critic module is introduced. Instead of generating entirely new solutions, the critic analyzes the current best solution and proposes targeted modifications. These modifications may involve reordering operations on a machine, relocating operations within a sequence, or reversing segments of operation sequences. The critic identifies inefficiencies such as idle times or conflicts and suggests localized improvements that can potentially reduce the overall objective value. This component introduces a structured reasoning loop into the optimization process, allowing the system to iteratively refine solutions.

2.5.4 Sequence Reconstruction using SSG

The modifications proposed by the critic are applied to machine-level operation sequences. Since these edits may lead to infeasible schedules, a Serial Schedule Generation (SSG) procedure is used to reconstruct a valid schedule. The SSG method schedules operations sequentially while ensuring that precedence constraints are respected and machine availability is maintained at all times. This guarantees that all candidate solutions produced after modification remain feasible.

2.5.5 Solution History and Selection Mechanism

The framework maintains a history of candidate solutions along with their corresponding objective values. This history serves as a memory that guides future solution generation. High-quality solutions are reused in prompts to the LLM, enabling informed exploration of the solution space. At the same time, the history is periodically pruned to retain only the best-performing candidates. This prevents the search process from becoming inefficient and ensures that the optimization remains focused on promising regions of the solution space.

2.5.6 Baseline Integration

A baseline solution generated using a classical optimization solver, such as OR-Tools CP-SAT, is incorporated into the framework. This baseline provides a reference point for evaluating improvements achieved through LLM-based optimization. It also ensures that the system begins with a feasible and reasonably good solution, allowing subsequent iterations to focus on refinement rather than exploration from scratch.

2.6 Experimental Setup

2.6.1 Model Configuration

The proposed framework is evaluated using multiple large language models of varying scales in order to analyze the impact of model capacity on optimization performance. The models used in this study include LLaMA 3.1 8B, LLaMA 3.3 70B, and GPT-OSS 120B. These models are accessed through API-based interfaces, ensuring consistent interaction through structured prompts. Each model is provided with identical problem descriptions and contextual inputs to maintain fairness across experiments.

2.6.2 Problem Instances and Initialization

The experiments are conducted on the set of combinatorial optimization problems described in Chapter 1. For each problem instance, the optimization process begins with the generation of an initial feasible solution. In cases where classical solvers are applicable, such as scheduling problems, a baseline solution is obtained using OR-Tools CP-SAT. This baseline serves as a reference point for evaluating improvements achieved by the proposed framework.

2.6.3 Solution Generation and Execution

The LLM is prompted to generate executable code that constructs candidate solutions. Unlike traditional approaches that directly output solutions, the generated code encodes

the logic required to build feasible schedules or assignments. This code is executed in a controlled environment, and the resulting output is parsed to extract the corresponding solution. Each candidate solution is then evaluated based on feasibility and objective value.

2.6.4 Evaluation Criteria

Feasibility is strictly enforced by verifying that all problem-specific constraints are satisfied. These constraints include precedence relationships, resource limitations, and structural requirements depending on the problem type. Any solution that violates these constraints is discarded. For feasible solutions, the objective value, such as makespan or total cost, is computed and used for comparison and ranking.

2.6.5 Iterative Optimization Process

The optimization process proceeds iteratively by combining both generative and reasoning-based refinement. In each iteration, the LLM generates new candidate solutions based on the current prompt, which includes previously successful solutions and accumulated feedback. Simultaneously, the critic module analyzes the best-known solution and proposes localized modifications to improve performance. These modifications are applied and reconstructed into feasible solutions using the scheduling procedure.

2.6.6 Solution History and Pruning

A solution history is maintained throughout the optimization process to store candidate solutions along with their objective values. This history is used to guide future generations by providing context to the LLM. To ensure computational efficiency, the history is periodically pruned to retain only the top-performing solutions. This prevents the search from becoming inefficient and keeps the optimization focused on promising regions of the solution space.

2.6.7 Termination Criteria

The optimization process terminates either after a fixed number of iterations or when convergence is observed. Convergence is defined as the absence of significant improvement in the objective value over successive iterations. The best solution identified during the process is reported as the final result.

2.6.8 Performance Metrics

The framework is evaluated using two primary metrics: feasibility and optimality. Feasibility ensures that the generated solutions satisfy all constraints, while optimality measures the quality of the solution with respect to the objective function. These metrics are used to compare performance across different models and problem instances.

2.7 Results

This section presents the experimental results obtained using the proposed OPRO + ReAct framework across different combinatorial optimization problems. For each problem, the performance of the framework is analyzed in terms of convergence behavior, feasibility, and optimality. The results are compared against known optimal solutions where available.

2.7.1 Classic 0/1 Knapsack Problem

The first experiment evaluates the performance of the proposed framework on the classic 0/1 Knapsack problem described in Chapter 1. The objective is to maximize the total value of selected items without exceeding the weight capacity.

Problem Instance Details:

Item	Weight	Value
Item 1	11	112
Item 2	8	116
Item 3	17	104
Item 4	19	104
Item 5	15	117
Item 6	12	53
Item 7	17	32
Item 8	9	51

Figures 2.2, 2.3, and 2.4 show the convergence behavior of different models across iterations.

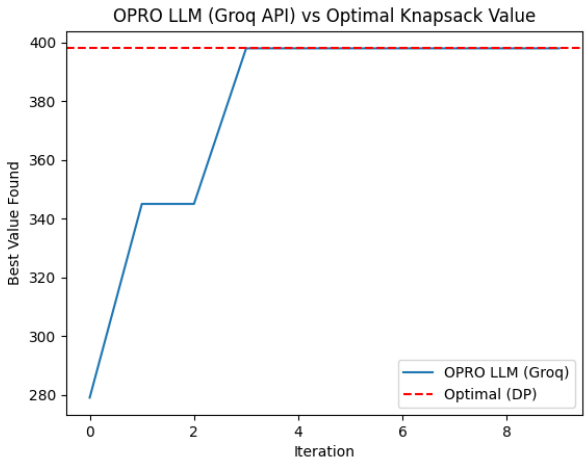

Figure 2.2: LLaMA 3.1 8B performance on 0/1 Knapsack.

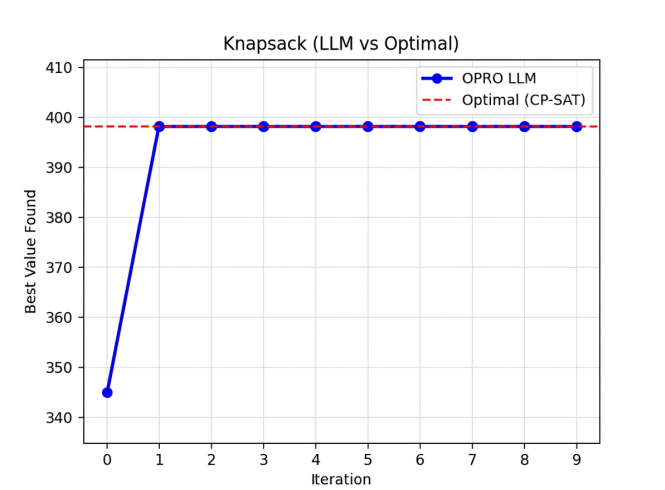

Figure 2.3: LLaMA 3.3 70B performance on 0/1 Knapsack.

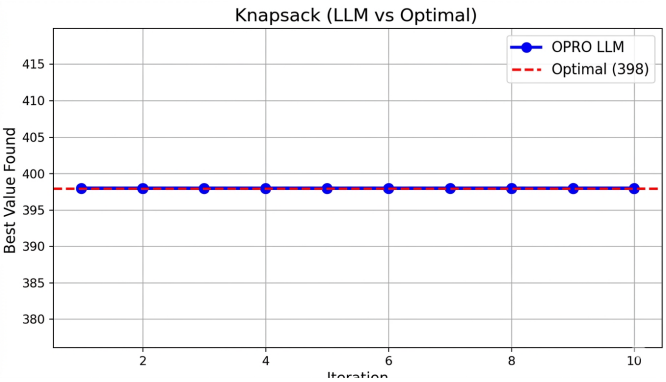

Figure 2.4: GPT-OSS 120B performance on 0/1 Knapsack.

Across all models, a consistent improvement in solution quality is observed over iterations. The smaller model initially starts with suboptimal solutions but is able to reach the optimal value after a few iterations. The larger models converge more rapidly and exhibit stable performance early in the process.

All models successfully reach the optimal value of 398, matching the dynamic programming baseline. The final solution corresponds to selecting Item1, Item2, Item5, and Item6, satisfying the capacity constraint while maximizing total value.

These results demonstrate that the proposed framework is robust across model sizes and is able to consistently identify optimal solutions for problems with relatively simple constraints.

2.7.2 Multi-constraint Knapsack Problem

The second experiment evaluates the framework on the multi-constraint knapsack problem, where each item must satisfy multiple resource constraints simultaneously.

Problem Instance Details:

Item	(w_1, w_2)	Value
Item 1	$(7, 20)$	73
Item 2	$(15, 11)$	69
Item 3	$(8, 7)$	30
Item 4	$(19, 11)$	42
Item 5	$(11, 4)$	85
Item 6	$(8, 3)$	67
Item 7	$(2, 12)$	31
Item 8	$(6, 2)$	117

Capacities: $(27, 33)$

Figures 2.5, 2.6, and 2.7 show the convergence behavior of different models.

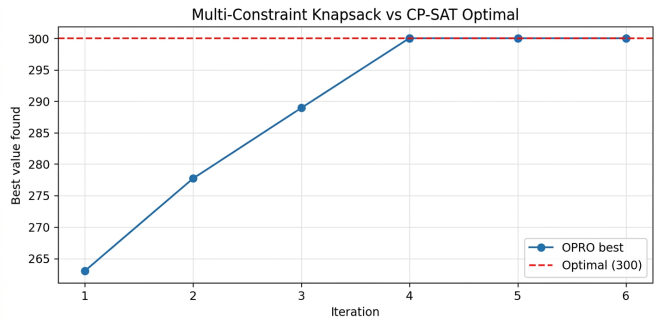

Figure 2.5: LLaMA 3.1 8B performance on MCKP.

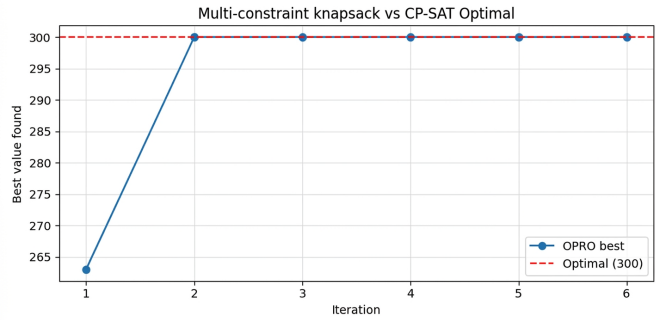

Figure 2.6: LLaMA 3.3 70B performance on MCKP.

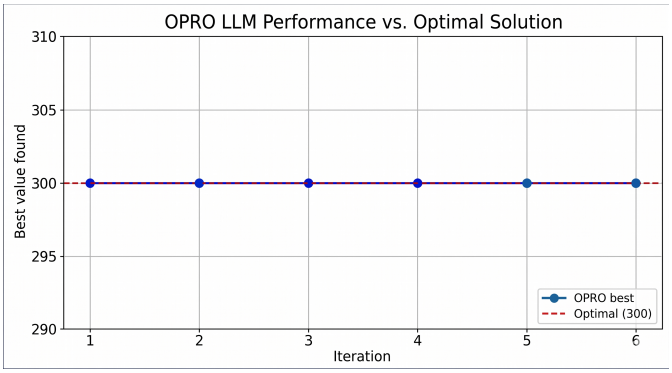

Figure 2.7: GPT-OSS 120B performance on MCKP.

All models successfully reach the optimal value of 300 while satisfying both constraints. However, differences are observed in convergence behavior. The smaller model shows gradual improvement over iterations before reaching optimality, whereas the larger models converge more rapidly, often identifying the optimal solution within fewer iterations.

The final solution corresponds to selecting Item5, Item6, Item7, and Item8, which satisfies both resource constraints while maximizing total value.

These results demonstrate that the proposed framework remains stable across different model sizes and is effective in handling multiple constraints, with larger models achieving faster convergence.

2.7.3 Bounded Knapsack Problem

The bounded knapsack problem introduces an additional constraint where each item can only be selected up to a specified quantity, increasing the combinatorial complexity compared to the standard knapsack problem.

Problem Instance Details:

Item	Weight	Value	Max Quantity
Item 1	9	43	2
Item 2	6	22	2
Item 3	10	41	1
Item 4	7	72	1
Item 5	9	21	2
Item 6	5	49	2

Capacity: 50

Optimal Value (CP-SAT): 300

Figures 2.8, 2.9, and 2.10 show the convergence behavior across models.

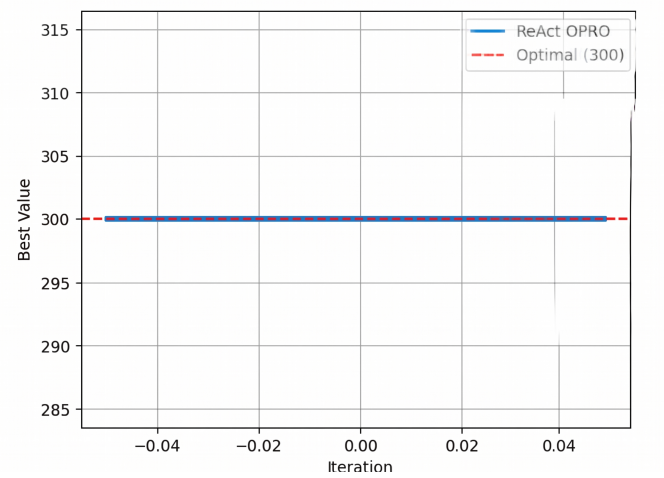

Figure 2.8: GPT-OSS 120B performance on bounded knapsack.

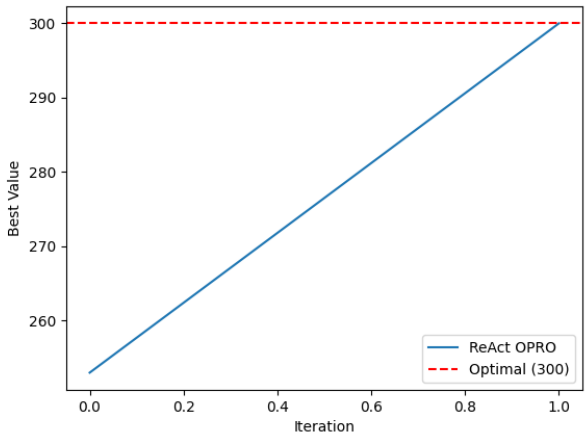

Figure 2.9: LLaMA 3.1 8B performance on bounded knapsack.

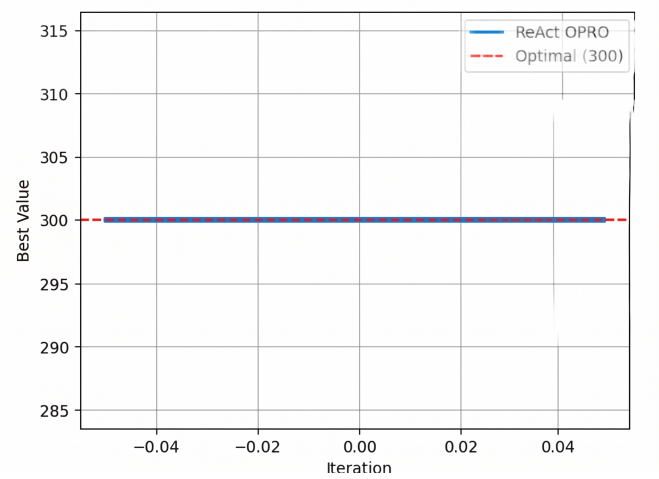

Figure 2.10: LLaMA 3.3 70B performance on bounded knapsack.

All models are able to reach the optimal value of 300; however, their convergence behavior differs significantly.

The GPT-OSS 120B and LLaMA 3.3 70B models converge to the optimal solution within a single iteration, indicating strong reasoning capability and direct generation of correct solution logic. In contrast, the LLaMA 3.1 8B model requires two iterations to reach the optimal value, initially producing suboptimal or partially incorrect solutions before improving in the subsequent iteration.

The logs further show that intermediate iterations may include invalid or incorrectly reconstructed solutions, particularly for smaller models.

The final optimal solution corresponds to the selection vector [2, 2, 0, 1, 0, 2], which satisfies both capacity and quantity constraints while maximizing total value.

These results highlight that while the proposed framework is capable of solving bounded optimization problems, model size plays a significant role in convergence speed and robustness.

2.7.4 Multi-Dimensional Knapsack Problem

This experiment evaluates the framework on a multi-dimensional knapsack problem involving both weight and volume constraints, increasing the complexity of feasibility compared to standard knapsack formulations.

Problem Instance Details:

Item	Weight	Volume	Value
Item 1	11	16	117
Item 2	8	28	59
Item 3	17	20	67
Item 4	19	20	31
Item 5	15	13	93
Item 6	12	17	89
Item 7	17	12	50
Item 8	9	11	62

Capacity: Weight = 50, Volume = 60

Optimal Value (CP-SAT): 361

Figures 2.11, 2.12, and 2.13 show the performance across models.

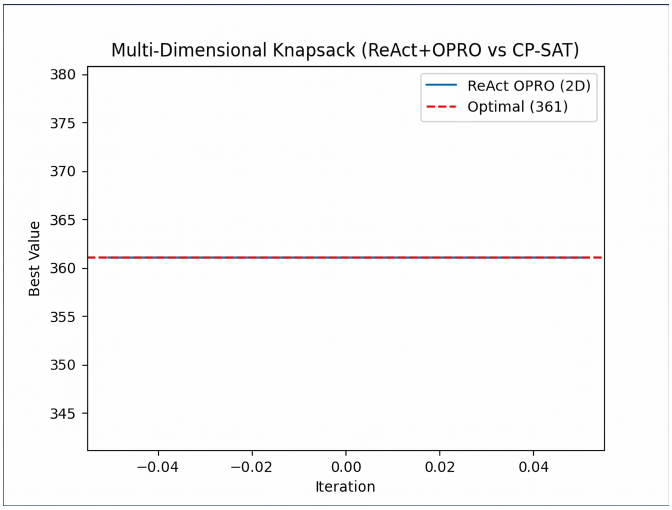

Figure 2.11: LLaMA 3.1 8B performance on multi-dimensional knapsack.

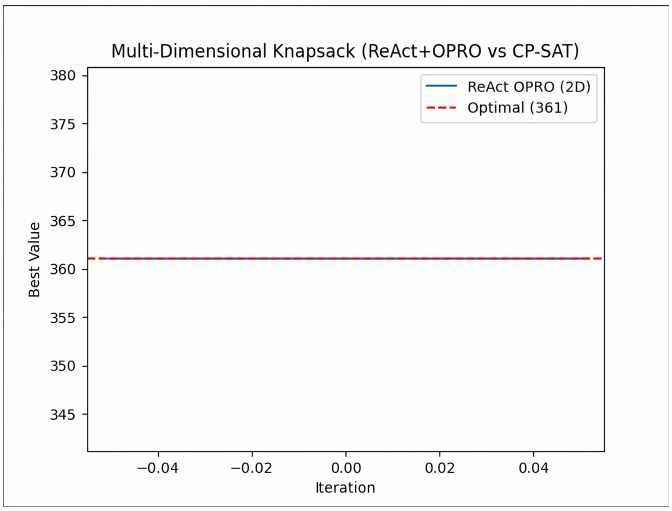

Figure 2.12: LLaMA 3.3 70B performance on multi-dimensional knapsack.

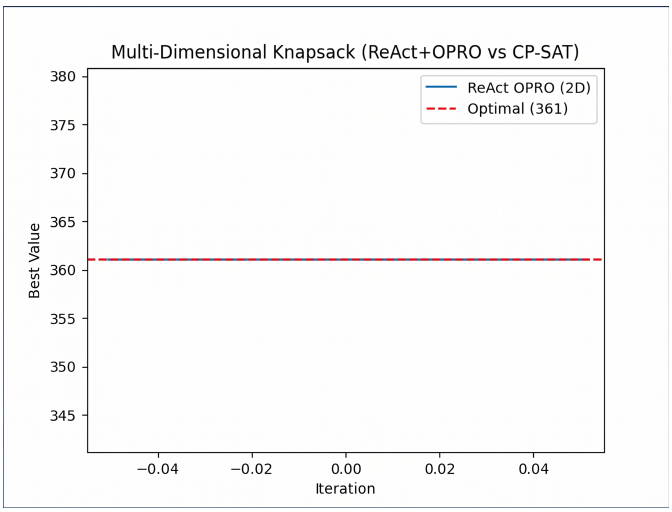

Figure 2.13: GPT-OSS 120B performance on multi-dimensional knapsack.

All models converge to the optimal value of 361 in a single iteration. The solution is directly obtained without requiring iterative refinement, indicating that the models are able to generate correct optimization logic from the initial prompt itself.

The final solution corresponds to selecting Item1, Item5, Item6, and Item8, which satisfies both weight and volume constraints while achieving the maximum value.

Although minor inconsistencies such as parsing errors or intermediate execution issues are observed in the logs, they do not affect the final outcome, as all models consistently recover and produce the optimal solution.

These results demonstrate that the proposed framework can effectively handle higher-dimensional constraints and is capable of directly generating optimal solutions even for more complex knapsack variants.

2.7.5 Multi-Objective Knapsack Problem

This experiment evaluates the framework on a multi-objective knapsack problem, where the goal is to maximize value while simultaneously minimizing risk under a weight constraint. Unlike previous problems, this formulation does not yield a single optimal solution, but instead produces a set of Pareto-optimal solutions representing trade-offs between objectives.

Problem Instance Details:

Item	Weight	Value	Risk
Item 1	11	112	30
Item 2	8	116	11
Item 3	17	104	33
Item 4	19	104	21
Item 5	15	117	39
Item 6	12	53	15
Item 7	17	32	11
Item 8	9	51	37

Capacity: Weight = 50

Figures 2.14, 2.15, and 2.16 illustrate the Pareto front generated by the models compared to the CP-SAT baseline.

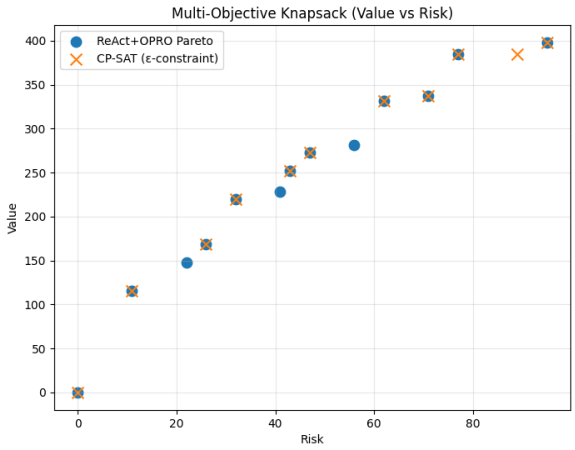

Figure 2.14: LLaMA 3.1 8B Pareto solutions (3 iterations).

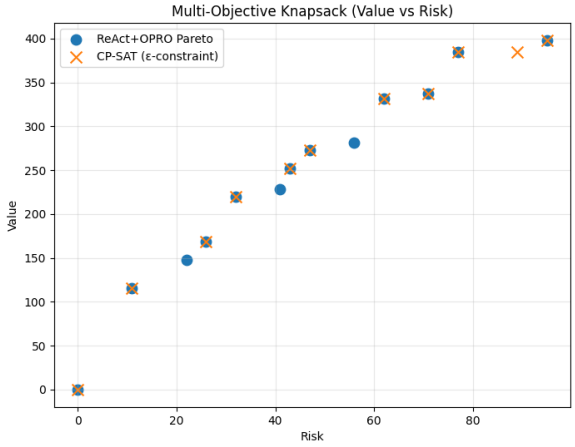

Figure 2.15: LLaMA 3.3 70B Pareto solutions (3 iterations).

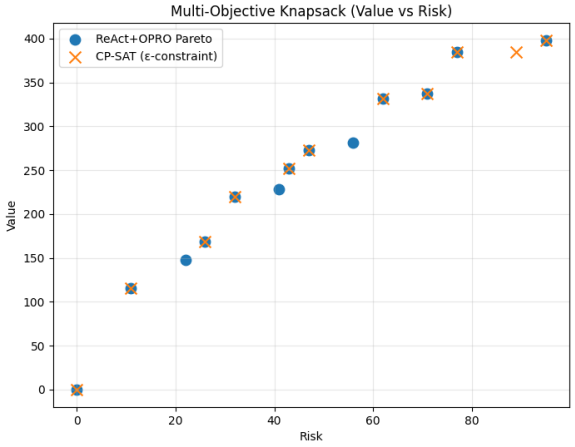

Figure 2.16: GPT-OSS 120B Pareto solutions (5 iterations).

The results show that, unlike single-objective problems, the framework does not converge to a single solution but instead attempts to construct a Pareto frontier. Across all models, multiple non-dominated solutions are generated, capturing different trade-offs between value and risk.

For LLaMA 8B and 70B, the framework stabilizes within three iterations, after which no significant improvement or expansion of the Pareto set is observed. In contrast, GPT-OSS 120B requires five iterations due to intermediate execution failures and inconsistent outputs before stabilizing.

A key observation is that the Pareto front generated by the models partially overlaps with the CP-SAT baseline but does not fully match it. Some high-quality trade-off solutions are correctly identified, particularly in the mid-range of risk, while certain extreme points are either missing or approximated.

Additionally, the logs reveal that early iterations often fail to produce valid Pareto sets, with empty outputs or incorrect code generation. Subsequent iterations correct these issues and successfully enumerate feasible solutions, indicating that the reasoning-feedback loop is essential for handling multi-objective formulations.

These results highlight a fundamental limitation of the framework: while it performs well for single-objective optimization, its ability to accurately and consistently approximate the full Pareto frontier remains limited. Multi-objective problems require more structured exploration and evaluation mechanisms, which are not fully captured by the current approach.

2.7.6 Travelling Salesman Problem (TSP)

To evaluate the performance of the framework on combinatorial routing problems, we consider the Travelling Salesman Problem (TSP), where the objective is to find the shortest possible tour that visits each city exactly once and returns to the starting point.

Each city is represented as a point in a 2D plane, where x and y denote the Cartesian coordinates of the city.

Problem instance:

City	x	y
City 0	51	74
City 1	92	74
City 2	14	87
City 3	71	99
City 4	60	23
City 5	20	2
City 6	82	21
City 7	86	52

The optimal route obtained using OR-Tools is: [0, 3, 1, 7, 6, 4, 5, 2]
with total distance: 310.00

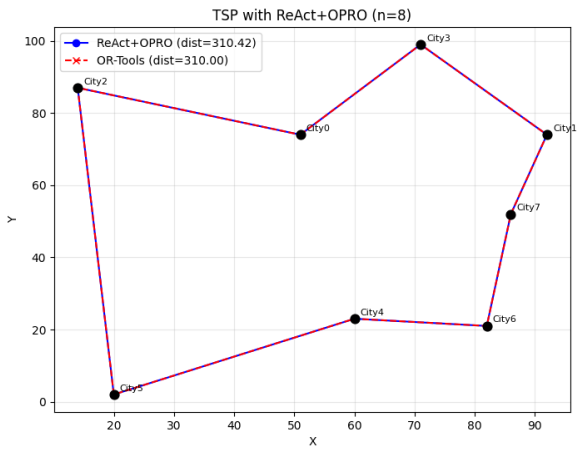

Figure 2.17: TSP solution using LLaMA 8B (3 iterations).

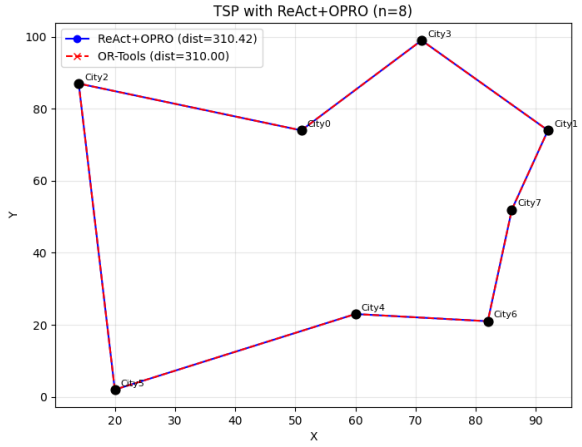

Figure 2.18: TSP solution using LLaMA 70B (2 iterations).

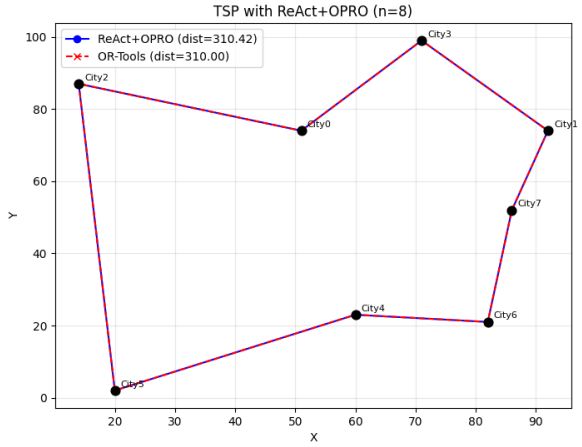

Figure 2.19: TSP solution using GPT-OSS 120B (2 iterations).

The ReAct+OPRO framework produces a route with total distance: 310.42

which is very close to the optimal solution, with a gap of 0.42. The generated tour structure closely matches the optimal route, differing only in minor edge selections because of the type difference between the code generated by the LLM and CP-SAT.

All models converge quickly, with LLaMA 8B requiring 3 iterations, while LLaMA 70B and GPT-OSS 120B converge in 2 iterations. After convergence, no further improvements are observed.

Unlike the multi-objective knapsack problem, TSP has a single optimal solution, allowing the framework to converge more reliably. However, some limitations are observed. Early iterations often produce invalid or incomplete outputs due to formatting

or execution errors, and many generated solutions rely on heuristic strategies such as nearest-neighbor or 2-opt. Despite multiple iterations, the framework does not exactly match the optimal solution, indicating limitations in fine-grained optimization.

Intermediate solutions are sometimes significantly worse before improving, which reflects instability in the reasoning process.

Overall, the framework is able to produce near-optimal solutions for TSP with very few iterations, but it lacks the precision required to consistently reach the exact optimal solution.

2.7.7 Vehicle Routing Problem (VRP)

To further evaluate the framework on multi-route combinatorial optimization problems, we consider the Capacitated Vehicle Routing Problem (VRP), where multiple vehicles must serve a set of customers while respecting capacity constraints and minimizing total travel distance.

Problem instance:

Node	x	y	Demand
0	50	50	0
1	51	87	6
2	92	99	25
3	14	23	37
4	71	2	16
5	60	21	26
6	20	52	29
7	82	1	31
8	86	87	32
9	74	29	20
10	74	37	19

Table 2.1: Node coordinates and demand for VRP instance (Node 0 is the depot)

Number of vehicles are 4 and Vehicle capacity is 80

The initial solution obtained using greedy seeding has total distance: 438.84

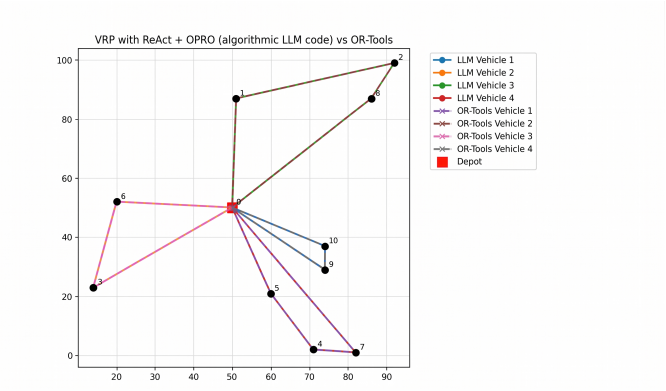

Figure 2.20: VRP solution using LLaMA 8B (3 iterations).

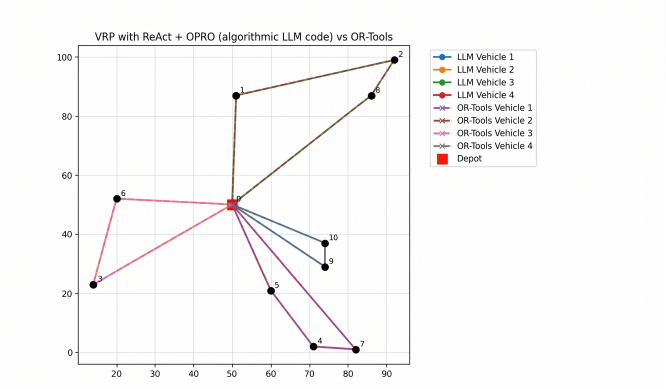

Figure 2.21: VRP solution using LLaMA 70B (3 iterations).

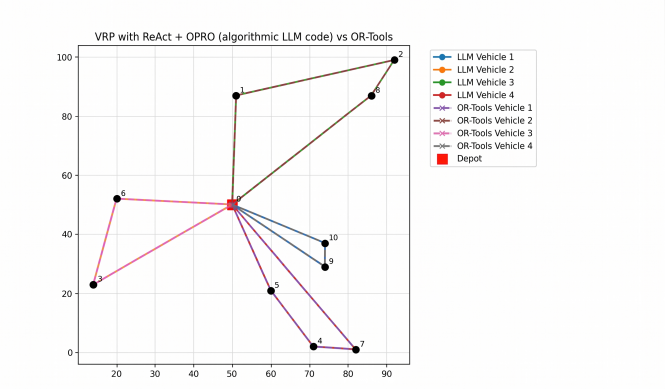

Figure 2.22: VRP solution using GPT-OSS 120B (3 iterations).

All three models (LLaMA 8B, LLaMA 70B, and GPT-OSS 120B) converge within 3 iterations. However, no improvement over the initial solution is observed, and the final distance remains: 438.84

The routes remain identical to the initial greedy solution across all iterations.

The framework is able to generate feasible routes that satisfy all capacity constraints, but it fails to improve upon the initial solution. The reasoning process largely focuses on assigning customers to vehicles rather than optimizing routes.

Logs also indicate that many generated solutions are infeasible or contain structural errors before filtering, highlighting instability in code generation.

Compared to TSP, the VRP introduces higher complexity due to multiple routes, capacity constraints, and interdependence between vehicle assignments. As a result, simple local heuristics such as greedy assignment are not sufficient, and the framework struggles to perform coordinated improvements across routes.

Overall, while the framework produces feasible solutions, it does not achieve meaningful optimization for VRP, revealing a limitation in handling structured multi-route problems.

2.7.8 Job-Shop Scheduling Problem (JSSP)

To further evaluate the capability of the framework on complex scheduling problems, we consider the Job-Shop Scheduling Problem (JSSP), where the objective is to minimize the makespan while respecting machine constraints and job precedence constraints.

Problem instance:

The job structure is defined as follows, where each job consists of a sequence of operations. Each operation specifies the machine on which it must be processed and its processing time.

Job	Operation	Machine	Processing Time
3*Job 0	Operation 1	Machine 0	6
	Operation 2	Machine 1	3
	Operation 3	Machine 2	9
3*Job 1	Operation 1	Machine 0	3
	Operation 2	Machine 2	9
	Operation 3	Machine 1	5
3*Job 2	Operation 1	Machine 1	5
	Operation 2	Machine 2	3
	Operation 3	Machine 0	7

Table 2.2: Job-Shop Scheduling instance showing operation sequence, machine assignment, and processing time

The optimal makespan obtained using OR-Tools is: 24

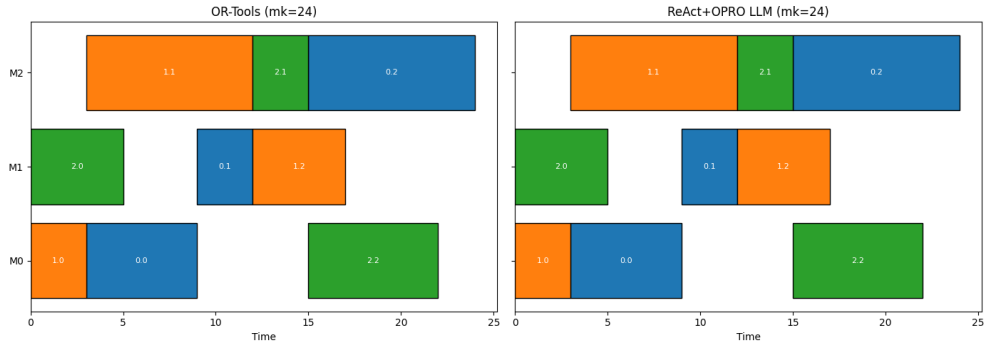

Figure 2.23: JSSP schedule using LLaMA 8B (8 iterations).

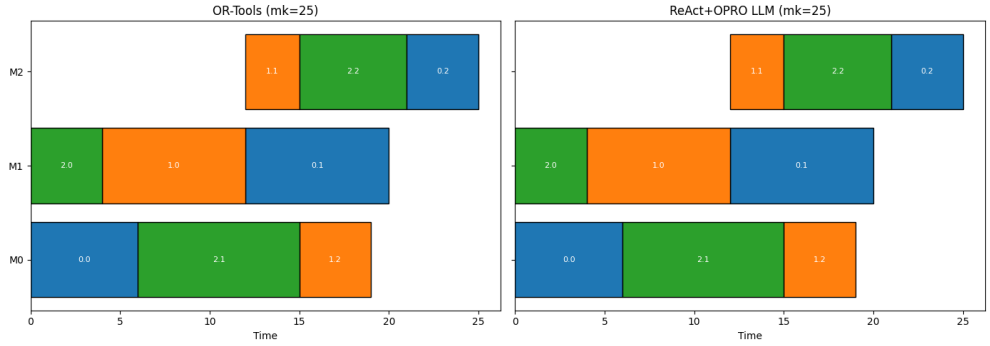

Figure 2.24: JSSP schedule using LLaMA 70B (6 iterations).

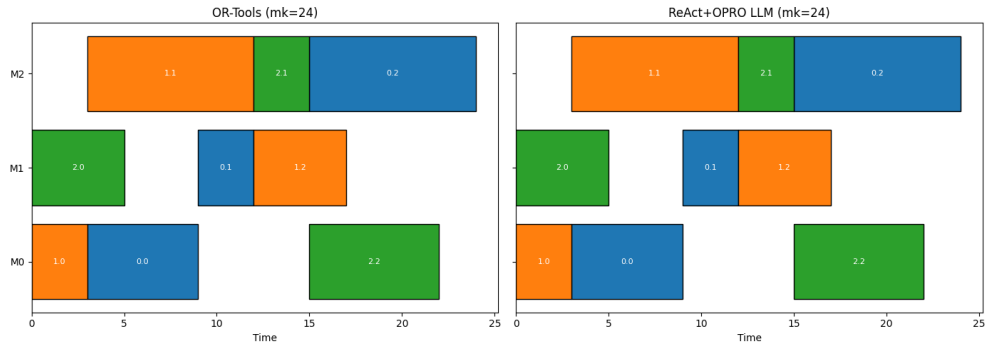

Figure 2.25: JSSP schedule using GPT-OSS 120B (3 iterations).

The ReAct+OPRO framework successfully identifies a schedule with makespan: 24

which exactly matches the optimal solution. The generated schedules are structurally identical to the OR-Tools solution, indicating that the framework is capable of solving small-scale scheduling problems optimally.

However, the convergence behaviour differs across models. GPT-OSS 120B converges fastest in 3 iterations, LLaMA 70B converges in 6 iterations, and LLaMA 8B requires 8 iterations to reach the optimal solution.

During early iterations, several issues are observed in the generated code, including syntax errors, recursion depth errors, and invalid schedule representations. These errors prevent feasible schedules from being produced in some iterations. Despite these challenges, the framework is able to recover and eventually converge to the optimal solution through iterative refinement.

Unlike simpler problems such as knapsack, JSSP requires strict adherence to both machine capacity and precedence constraints, making it significantly more sensitive to implementation errors. The iterative correction mechanism plays a crucial role in overcoming these failures.

Overall, the framework demonstrates the ability to reach optimal solutions for JSSP, but with slower and less stable convergence compared to simpler combinatorial problems.

2.7.9 Bin Packing Problem

To evaluate the framework on packing and allocation problems, we consider the Bin Packing Problem, where the objective is to minimize the number of bins required to store all items without exceeding bin capacity.

Problem instance:

```
Items: [8, 5, 14, 12, 9, 14, 6, 8, 11, 4, 8, 12]
Bin capacity: 20
```

The optimal solution obtained using OR-Tools requires:

6 bins

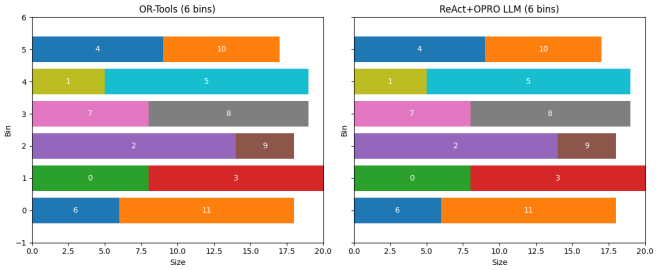

Figure 2.26: Bin packing using LLaMA 8B (6 iterations).

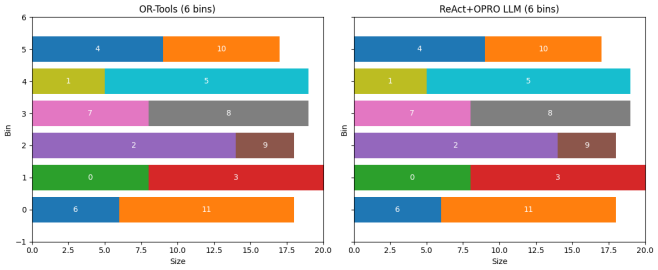

Figure 2.27: Bin packing using LLaMA 70B (6 iterations).

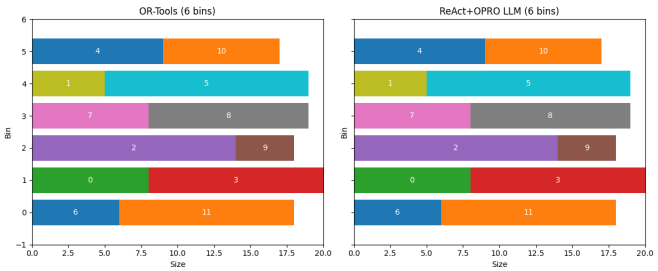

Figure 2.28: Bin packing using GPT-OSS 120B (6 iterations).

The ReAct+OPRO framework successfully identifies a solution using 6 bins, which matches the optimal solution obtained using OR-Tools.

However, the convergence behaviour differs significantly across models. GPT-OSS 120B generates a correct and feasible solution in the first iteration itself and maintains the same solution across subsequent iterations. LLaMA 70B also produces feasible solutions early and remains stable throughout the iterations. In contrast, LLaMA 8B exhibits unstable behaviour, producing multiple invalid packings and execution errors in early iterations before eventually converging to a feasible solution.

Across iterations, no improvement beyond 6 bins is observed, as the optimal solution is already achieved. The framework primarily explores different equivalent packings rather than reducing the number of bins.

This behaviour indicates that while the framework is capable of identifying optimal solutions for bin packing, smaller models struggle with generating correct and executable code, leading to instability. Larger models, on the other hand, demonstrate more reliable reasoning and faster convergence.

Overall, the framework performs well on bin packing in terms of optimality, but its effectiveness depends strongly on model capability, particularly for maintaining feasibility during iterative refinement.

2.7.10 Assignment Problem

We evaluate the Assignment Problem using a 6×6 cost matrix, where each worker must be assigned to exactly one job such that the total cost is minimized. The optimal solution is obtained using the Hungarian algorithm as a reference baseline.

The cost matrix used in the experiment is shown below:

	Job 0	Job 1	Job 2	Job 3	Job 4	Job 5
Worker 0	43	33	19	47	12	25
Worker 1	43	23	27	15	15	28
Worker 2	40	44	28	7	26	6
Worker 3	28	48	34	42	6	25
Worker 4	37	16	26	48	29	31
Worker 5	46	32	20	19	48	7

Each row represents a worker, and each column represents a job. The value at position (i, j) indicates the cost of assigning Worker i to Job j. For example, assigning Worker 0 to Job 2 has a cost of 19, while assigning Worker 3 to Job 4 has a cost of 6. The goal is to select exactly one entry from each row and each column such that the total cost is minimized.

The optimal assignment cost obtained using the Hungarian method is 92. We compare this with the solution generated by the ReAct+OPRO framework using different LLMs.

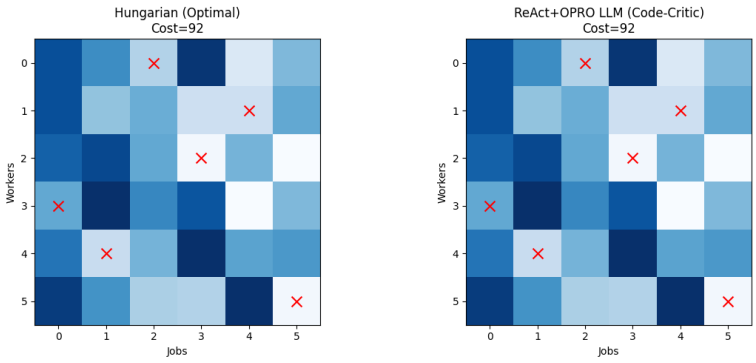

Figure 2.29: Assignment Problem: Optimal (Hungarian) vs ReAct+OPRO LLM solution

From the results, all models (LLaMA 8B, LLaMA 70B, and GPT-OSS 120B) successfully find the optimal assignment with total cost equal to 92, matching the Hungarian solution exactly. The selected assignments (marked by red crosses) are identical in both the optimal and LLM-generated solutions, indicating that the framework is able to recover the globally optimal matching.

In terms of iteration behavior, all models were executed for a fixed budget of 6 iterations. However, the optimal solution is already discovered in the first iteration itself,

and subsequent iterations do not improve the solution further. Instead, later iterations explore alternative implementations of the assignment solver, some of which are invalid due to coding errors, while others produce equivalent optimal solutions.

Overall, this experiment demonstrates that for structured combinatorial problems like the Assignment Problem, the ReAct+OPRO framework is capable of quickly identifying optimal solutions. The additional iterations primarily serve as redundant exploration rather than improving solution quality.

2.7.11 Generalized Assignment Problem (GAP)

We evaluate the Generalized Assignment Problem (GAP), where multiple agents must be assigned a set of tasks while respecting capacity constraints. Each assignment incurs a cost and consumes a certain amount of resource (usage). The objective is to minimize total cost while ensuring that no agent exceeds its capacity.

The instance used in our experiment consists of 4 agents and 8 tasks.

Cost Matrix (agents $\times$ tasks):

	$T0$	$T1$	$T2$	$T3$	$T4$	$T5$	$T6$	$T7$
$A0$	11	24	19	15	12	25	11	23
$A1$	27	15	15	28	25	8	12	28
$A2$	7	26	25	6	28	16	10	6
$A3$	25	5	16	26	16	29	21	14

Usage Matrix (resource consumption):

	$T0$	$T1$	$T2$	$T3$	$T4$	$T5$	$T6$	$T7$
$A0$	3	7	4	9	3	5	3	7
$A1$	5	9	7	2	4	9	2	9
$A2$	5	2	4	7	8	3	1	4
$A3$	2	8	4	2	6	6	4	6

Agent Capacities:

$$[16,\ 24,\ 16,\ 24]$$

Each task must be assigned to exactly one agent. For example, assigning task $T3$ to agent $A2$ incurs cost 6 and consumes 7 units of capacity.

The greedy baseline produces a feasible solution with total cost 70, which serves as the reference.

Task	Greedy Assignment (Agent)	ReAct + OPRO Assignment (Agent)
T0	0	0
T1	0	0
T2	1	1
T3	1	1
T4	2	2
T5	2	2
T6	2	2
T7	3	3

Table 2.3: GAP assignment comparison between Greedy and ReAct+OPRO approaches

From the figure, each task (T0–T7) is mapped to an agent (A0–A3). Blue circles denote the greedy assignment, while red crosses denote the ReAct+OPRO solution. The overlap between markers indicates that both methods produce identical assignments for all tasks.

This confirms that the LLM-based approach successfully recovers the same optimal (or near-optimal) solution as the greedy baseline with total cost 70.

In terms of iteration behavior, the results are obtained from execution logs of all three models:

- LLaMA 8B: 6 iterations
- LLaMA 70B: 6 iterations
- GPT-OSS 120B: 6 iterations

Although all models run for 6 iterations, the optimal solution is already discovered in the first iteration itself, as shown in the logs (“Best so far: cost = 70”) :contentReference[oaicite:0]index=0. Subsequent iterations primarily explore alternative imple-

mentations, including brute-force enumeration and DFS-based approaches. Some intermediate iterations produce invalid or infeasible solutions due to coding errors, but the framework consistently retains the best feasible solution found so far.

Overall, this experiment highlights that the ReAct+OPRO framework is capable of solving GAP efficiently, matching the baseline solution in the very first iteration, while later iterations contribute to exploration rather than improvement.

2.7.12 Maximum Flow and Minimum Cut

We consider a directed graph with 8 nodes, where node 0 is the *source* and node 7 is the *sink*. Each directed edge (u, v) has a capacity, representing the maximum amount of flow that can be sent along that edge.

The objective of the Maximum Flow problem is to send as much flow as possible from the source to the sink, while respecting:

- Capacity constraints on each edge
- Flow conservation at intermediate nodes

The generated capacity matrix is shown below:

	0	1	2	3	4	5	6	7
0	0	0	8	0	20	0	0	0
1	15	0	0	11	8	0	7	19
2	0	0	0	11	0	0	11	0
3	0	0	4	0	0	0	8	3
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	6
6	12	0	6	2	1	0	0	0
7	0	0	12	0	0	0	0	0

Each entry (i, j) denotes the capacity of the edge from node i to node j. A value of 0 means no direct edge exists.

The baseline solution using Edmonds–Karp gives:

- Maximum flow = 3
- Minimum cut edge = $(3 \rightarrow 7)$ with capacity 3

The minimum cut represents the bottleneck of the network. Removing this edge disconnects the source from the sink, and its capacity equals the maximum flow.

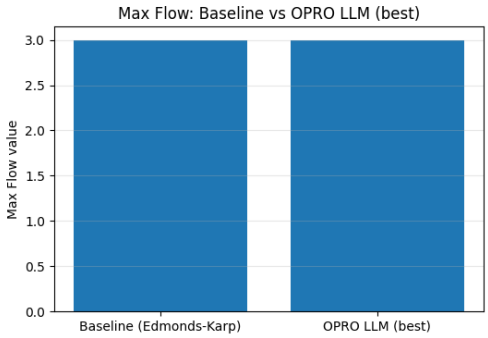

Figure 2.30: Maximum Flow Comparison

Both the baseline and the ReAct+OPRO approach achieve the same optimal flow value.

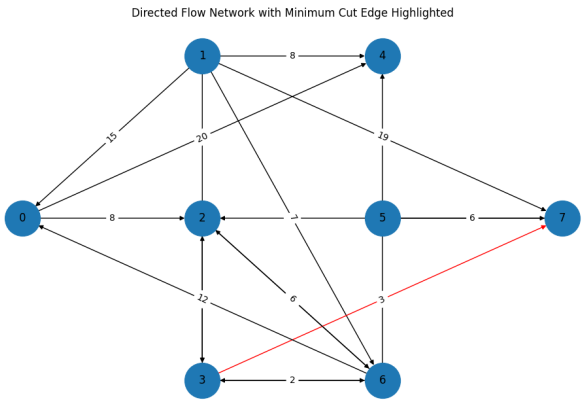

Figure 2.31: Graph with Minimum Cut Highlighted

The red dashed edge corresponds to the minimum cut, clearly identifying the limiting constraint in the network.

All models converge in:

- LLaMA 8B: 6 iterations
- LLaMA 70B: 6 iterations
- GPT-OSS 120B: 6 iterations

2.7.13 Minimum-Cost Flow

We now extend the problem by associating a cost with each edge, in addition to capacity. The objective is to send flow from source to sink such that:

- The total flow is maximized
- The total cost of sending this flow is minimized

Capacity Matrix

	0	1	2	3	4	5	6	7
0	0	0	7	5	3	0	0	0
1	5	0	0	7	3	0	3	8
2	0	0	0	4	0	0	8	0
3	0	0	6	0	0	0	2	4
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	6
6	8	0	5	4	6	4	0	0
7	0	0	1	3	0	0	0	0

Cost Matrix

	0	1	2	3	4	5	6	7
0	0	0	4	15	8	0	0	0
1	5	0	0	10	7	0	11	5
2	0	0	0	8	0	0	3	0
3	0	0	5	0	0	0	8	14
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	2
6	12	0	1	10	13	9	0	0
7	0	0	11	15	0	0	0	0

Here:

- Capacity(i, j) limits how much flow can pass
- Cost(i, j) represents the cost per unit flow

Thus, sending flow along different paths results in different total costs.

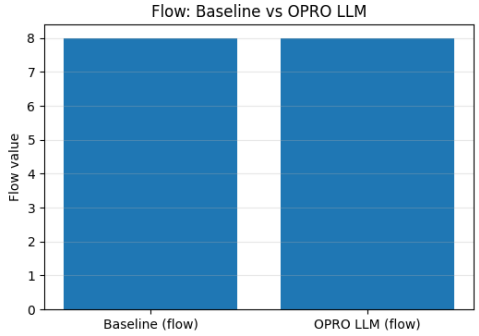

Figure 2.32: Flow Comparison

Both the baseline and the ReAct+OPRO method achieve identical flow values.

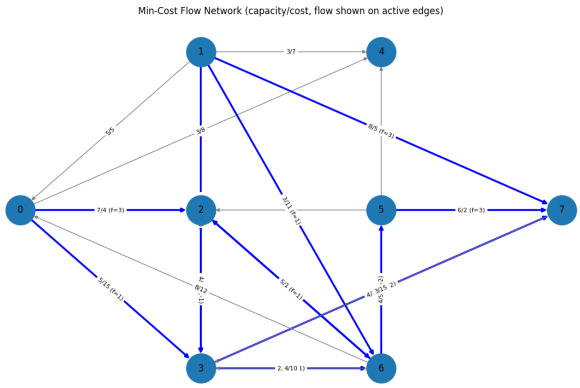

Figure 2.33: Min-Cost Flow Visualization

In the visualization:

- Each edge is labeled as $(capacity/cost)$
- Highlighted edges show the selected flow
- Flow values are annotated along edges

The LLM solution matches both the flow value and total cost of the optimal solution.

All models converge in:

- LLaMA 8B: 6 iterations
- LLaMA 70B: 6 iterations
- GPT-OSS 120B: 6 iterations

2.8 Comparative Analysis

We compare the performance of the standard OPRO framework with the proposed OPRO + ReAct approach across all optimization problems considered in this chapter.

The performance comparison indicates that while the OPRO method exhibits inconsistent results, the combination of OPRO + ReAct achieves consistent performance.

Across all experiments, a clear distinction is observed between the two approaches. The standard OPRO framework often exhibits unstable behavior, particularly in problems involving multiple constraints or structured solution spaces. In several cases, the

generated solutions violate feasibility conditions or fail to improve consistently across iterations.

In contrast, the integration of ReAct into the optimization loop leads to a substantial improvement in performance. The resulting framework demonstrates significantly higher stability, improved correctness of generated solutions, and a noticeable reduction in infeasible outputs. Importantly, even smaller models such as LLaMA 8B benefit from this integration and are able to achieve competitive performance.

While both approaches perform adequately on simpler problems such as knapsack and assignment, the difference becomes more pronounced in complex settings such as scheduling, routing, and multi-objective optimization. In these cases, OPRO alone struggles to maintain feasibility, whereas OPRO + ReAct consistently produces valid and improving solutions. These results indicate that reasoning-based feedback is essential for reliable optimization using LLMs.

2.9 Discussion

The experimental results highlight the importance of incorporating structured reasoning into the optimization process. The addition of ReAct enables the model to move beyond simple score-based improvements and instead engage in iterative refinement guided by feasibility and correctness.

A key advantage of this approach is improved constraint satisfaction. By explicitly evaluating whether generated solutions meet problem requirements, the model is able to identify violations and correct them in subsequent iterations. This leads to a more reliable optimization process, particularly in problems where feasibility is critical.

Furthermore, the reasoning mechanism introduces a more structured search process. Instead of generating independent candidate solutions, the model incrementally improves its outputs based on feedback from previous iterations. This reduces randomness and stabilizes convergence behavior.

In contrast, standard OPRO relies primarily on objective score feedback, which does not capture structural correctness. As a result, solutions may appear numerically improved while still being infeasible. The ReAct extension addresses this limitation by

enabling error detection, correction, and iterative refinement of solution logic.

These improvements are especially evident in scheduling and routing problems, where small implementation errors can invalidate entire solutions. The ability to debug and refine generated code plays a critical role in achieving feasible outcomes.

Overall, the results suggest that combining optimization-by-prompting with reasoning significantly enhances the capability of LLMs as optimizers.

2.10 Limitations and Open Questions

Although the proposed framework performs well on benchmark problems, its effectiveness on large-scale, tightly constrained optimization problems remains an open question.

Several limitations are observed from the experiments. The framework struggles in problems with complex interdependent constraints, such as vehicle routing, where coordinated improvements across multiple routes are required. Similarly, in multi-objective settings, the generated solutions only partially approximate the Pareto frontier, indicating limitations in structured exploration.

Another important challenge is the instability observed in early iterations, where generated code may be invalid or non-executable. While the framework is able to recover through iterative refinement, this behavior highlights the dependency on correct code generation.

More broadly, the current evaluation is limited to relatively small problem instances. Real-world optimization problems, such as large-scale scheduling and routing, introduce significantly higher complexity due to scale, dynamic constraints, and uncertainty. In such settings, maintaining feasibility and scalability becomes substantially more challenging.

Understanding whether LLM-based optimization can generalize effectively to these real-world scenarios remains an important direction for future research.

2.11 Chapter Summary

In this chapter, we evaluated the OPRO framework and its extension with ReAct across a diverse set of combinatorial optimization problems.

The results show that integrating ReAct leads to more stable and reliable optimization behavior. The framework is able to achieve optimal or near-optimal solutions across multiple problem classes, with significantly improved feasibility compared to standard OPRO. While larger models converge faster, smaller models also benefit substantially from reasoning-based feedback.

At the same time, limitations are observed in handling complex structured problems such as routing and scheduling, where the framework struggles to perform coordinated improvements.

These findings motivate further investigation into more realistic and challenging applications. In the next chapters, we extend this framework to hospital scheduling and flight crew assignment problems, which provide a stronger test of scalability and robustness.

2.12 Model-wise Observations

The experimental results reveal consistent differences in behavior across model sizes. The LLaMA 3.1 8B model typically requires multiple iterations to reach optimal solutions and is more sensitive to prompt structure. It frequently produces invalid or incomplete outputs in early iterations, although it benefits significantly from the ReAct correction mechanism.

The LLaMA 3.3 70B model demonstrates improved stability and better adherence to constraints. It is able to achieve near-optimal or optimal solutions in fewer iterations and performs consistently across most problem types, although occasional plateaus in improvement are observed.

The GPT-OSS 120B model exhibits the strongest performance overall. It often converges within very few iterations and produces highly consistent and feasible so-

lutions. Compared to smaller models, it generates significantly fewer invalid outputs and demonstrates stronger reasoning capabilities.

Overall, model size correlates with convergence speed and robustness. However, the introduction of ReAct reduces this gap, enabling smaller models to perform significantly better than with OPRO alone. These observations align with the trends seen across all experimental results.

CHAPTER 3

Hospital Scheduling Problem

3.1 Introduction

Hospital scheduling is a critical operational problem that involves assigning medical staff to patients while satisfying a variety of temporal, resource, and regulatory constraints. Unlike standard scheduling problems, hospital environments require strict adherence to feasibility conditions such as time compatibility, department specialization, and legal duty limits. Even small violations in these constraints can lead to infeasible or unsafe schedules.

From an optimization perspective, this problem is combinatorial in nature and closely related to crew scheduling and job-shop scheduling. The complexity arises due to the need to simultaneously satisfy sequencing constraints, resource limits, and operational feasibility conditions, while minimizing the total number of doctors required.

Classical optimization formulations of such problems, when solved directly using standard integer programming techniques, often become computationally expensive due to the exponential growth of the solution space. To address this challenge, advanced decomposition-based methods such as column generation, cutting plane methods, and Benders decomposition have been developed. These approaches significantly improve computational efficiency by decomposing the problem into smaller subproblems, reducing the effective search space, and enabling faster convergence to optimal solutions.

However, even these advanced methods rely on carefully designed subroutines for pricing, cut generation, and feasibility checking. Designing these components for real-world constraints remains complex, computationally intensive, and requires significant domain expertise. In particular, the effectiveness of these methods depends heavily on the quality of heuristics used within their iterative procedures.

This motivates the exploration of whether large language models can assist in these optimization pipelines. Recent advances in LLMs suggest that they can generate struc-

tured reasoning and executable code, making them potential candidates for guiding combinatorial search processes. Instead of manually designing heuristics for each subproblem, LLMs can potentially learn to propose feasible schedules, detect constraint violations, and iteratively refine solutions.

In this chapter, we investigate whether large language models, integrated through the OPRO + ReAct framework, can assist or enhance classical optimization methods. The central question is whether reasoning-driven LLMs can further improve the efficiency, scalability, and adaptability of decomposition-based algorithms while maintaining solution quality and feasibility.

3.2 Problem Formulation

A hospital operates a set of daily medical procedures. Each patient requires exactly one doctor. A doctor can perform multiple procedures in sequence, provided that the following operational and medical constraints are satisfied:

1. Patients are time-compatible, i.e., the start time of the next procedure must be after the completion of the previous one.
2. The total duty time of a doctor does not exceed legal working limits (maximum 8 hours).
3. A minimum rest or transition time between consecutive procedures is maintained (at least 1 hour).
4. Each doctor must be assigned only to patients belonging to their respective medical department.
5. Transition times between departments are respected, such as ICU to surgery requiring 10 minutes and surgery to ward requiring 5 minutes.
6. Each patient must be treated before their respective deadline.

The objective is to assign doctors to patients such that every patient is treated exactly once, all operational and medical constraints are satisfied, and the total number of doctors used is minimized.

This problem can be interpreted as a combinatorial scheduling problem where feasible solutions correspond to valid sequences of patient assignments (doctor schedules) that satisfy all constraints simultaneously.

3.3 Mathematical Model

We now present a mathematical formulation of the hospital scheduling problem.

Let P denote the set of patients and D denote the set of available doctors. Each patient $i \in P$ is associated with a processing time t_i, a department requirement d_i, and a deadline δ_i. Each doctor has a maximum allowable working time of $T = 8$ hours.

We define a binary decision variable x_{ij} such that:

$$x_{ij} = \begin{cases} 1, & \text{if patient } i \text{ is assigned to doctor } j \\ 0, & \text{otherwise} \end{cases}$$

The objective is to minimize the number of doctors used. This can be modeled by introducing an additional binary variable y_j indicating whether doctor j is active:

$$\min \sum_{j \in D} y_j$$

subject to the following constraints:

Each patient must be assigned exactly once:

$$\sum_{j \in D} x_{ij} = 1 \quad \forall i \in P$$

Assignment consistency constraint:

$$x_{ij} \leq y_j \quad \forall i \in P, \forall j \in D$$

Total working time constraint for each doctor:

$$\sum_{i \in P} t_i \cdot x_{ij} \leq T \quad \forall j \in D$$

Department compatibility constraint:

$$x_{ij} = 0 \quad \text{if doctor } j \text{ cannot serve department } d_i$$

In addition to these constraints, sequencing feasibility must be enforced. For any pair of patients assigned to the same doctor, their execution order must respect time compatibility, minimum rest periods, and transition times. These constraints are combinatorial in nature and are typically handled using decomposition methods rather than being explicitly enumerated in the formulation.

This model forms the basis for the optimization approaches discussed in the next section, where we explore classical decomposition techniques and their LLM-enhanced variants.

3.4 Baseline Optimization Approaches

This section presents the classical optimization techniques used to solve the hospital scheduling problem. These methods serve as the baseline against which the proposed OPRO + ReAct framework is evaluated. Each approach is designed to handle combinatorial complexity and constraint satisfaction, but differs in how the solution space is explored and refined.

3.4.1 Column Generation Approach

Column Generation (Pal 2005) is a decomposition-based optimization technique used for large-scale integer programming problems. Instead of enumerating all possible schedules upfront, the method starts with a restricted subset and iteratively introduces new variables that can improve the solution.

In the context of hospital scheduling, each column represents a feasible schedule assigned to a doctor, consisting of a sequence of patient visits that satisfy all operational constraints such as time compatibility, duty limits, and department matching.

The approach begins by initializing the master problem with simple schedules, typ-

ically assigning one patient per doctor. This restricted master problem (RMP) is solved as a linear program to obtain a fractional solution along with dual prices associated with each constraint.

Using these dual values, a pricing subproblem is solved to identify whether there exists a new schedule with negative reduced cost. This subproblem is typically modeled as a resource-constrained shortest path problem, where the objective is to construct a feasible sequence of patients that improves the current solution.

If such a schedule is found, it is added to the master problem as a new column, and the process repeats. Otherwise, the algorithm terminates, and the final integer solution is obtained by solving the master problem with integrality constraints.

Mathematical Formulation Let $\mathcal{C}$ denote the set of all feasible schedules (columns), where each column $c \in \mathcal{C}$ represents a valid sequence of patients assigned to a doctor. Let $a_{ic} = 1$ if patient i is included in column c, and 0 otherwise.

We define a decision variable x_c indicating whether schedule c is selected.

$$\min \sum_{c \in \mathcal{C}} x_c$$

subject to:

$$\sum_{c \in \mathcal{C}} a_{ic} x_c = 1 \quad \forall i \in P$$

$$x_c \geq 0 \quad \forall c \in \mathcal{C}$$

Since the number of feasible schedules is exponential, the problem is solved using a restricted master problem. The reduced cost of a column is given by:

$$\bar{c}_c = 1 - \sum_{i \in P} \pi_i a_{ic}$$

where π_i are the dual variables. A new column is added if $\bar{c}_c < 0$.

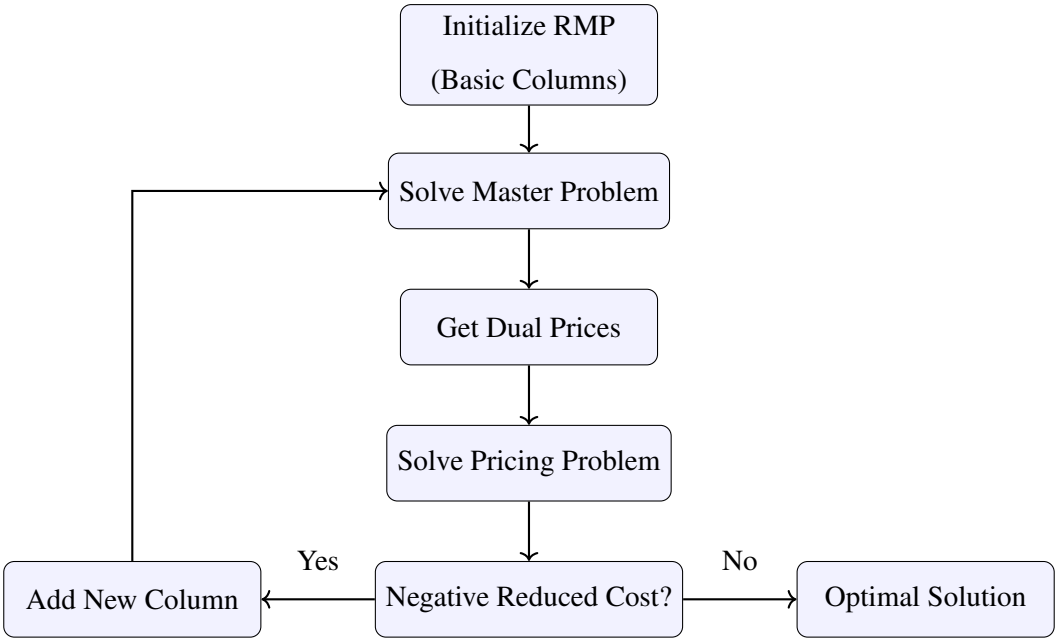

Figure 3.1: Column Generation framework for hospital scheduling. The algorithm iteratively generates feasible schedules (columns) using a pricing subproblem until no improving column is found.

3.4.2 Cutting Plane Approach

The Cutting Plane (Bùi and Marchi 2025) method solves integer optimization problems by iteratively refining a relaxed linear formulation. Instead of enforcing all constraints explicitly from the beginning, the method starts with a relaxed problem and progressively adds violated constraints (cuts) to eliminate infeasible solutions.

In this setting, the hospital scheduling problem is first formulated as a linear program where decision variables may take fractional values. The initial solution may violate combinatorial constraints such as assigning multiple overlapping patients to a single doctor.

At each iteration, the current solution is analyzed to detect violations. Based on this analysis, additional constraints are generated to cut off the infeasible region of the solution space. These constraints are then added to the model, and the linear program is re-solved.

This process continues until no violated constraints remain, at which point the solution becomes feasible with respect to all problem requirements. A final integer solution is then extracted.

Mathematical Formulation The problem is initially relaxed as:

$$\min c^T x$$

$$Ax \leq b, \quad x \geq 0$$

At each iteration, violated constraints of the form:

$$\alpha^T x \leq \beta$$

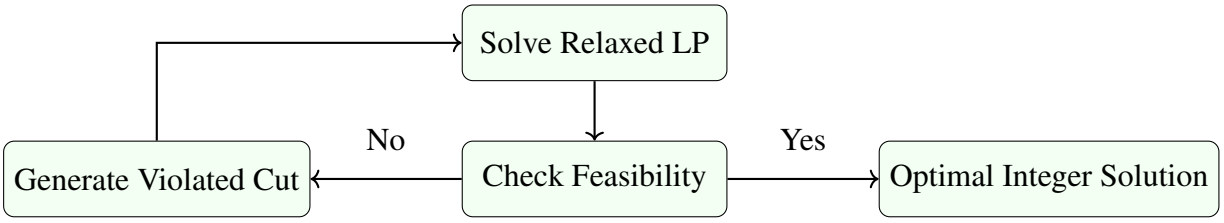

Figure 3.2: Cutting Plane method. The algorithm iteratively detects violated constraints and refines the feasible region until all constraints are satisfied.

3.4.3 Benders Decomposition Approach

Benders Decomposition (Ciré, Çoban, and Hooker 2016) is used for problems that can be separated into a master problem and one or more subproblems. It is particularly useful when the problem contains complicating constraints that can be isolated.

For the hospital scheduling problem, the master problem determines the assignment of patients to doctors, while the subproblem checks whether the assigned schedules satisfy feasibility constraints such as time ordering, rest requirements, and department compatibility.

The algorithm begins by solving the master problem, which provides a tentative assignment. This assignment is then evaluated in the subproblem. If the solution is feasible, it is accepted as optimal. Otherwise, infeasibility information is used to generate Benders cuts, which are added to the master problem to eliminate invalid assignments.

The process iterates between solving the master problem and generating cuts until

a feasible and optimal solution is obtained.

Mathematical Formulation The master problem determines assignment variables:

$$\min \sum_{j \in D} y_j$$

subject to assignment constraints.

The subproblem checks feasibility of schedules under sequencing and resource constraints. If infeasible, a Benders cut of the form:

$$\theta \geq \alpha^T y + \beta$$

is generated and added to the master problem.

This iterative process continues until no infeasibility is detected and an optimal solution is obtained.

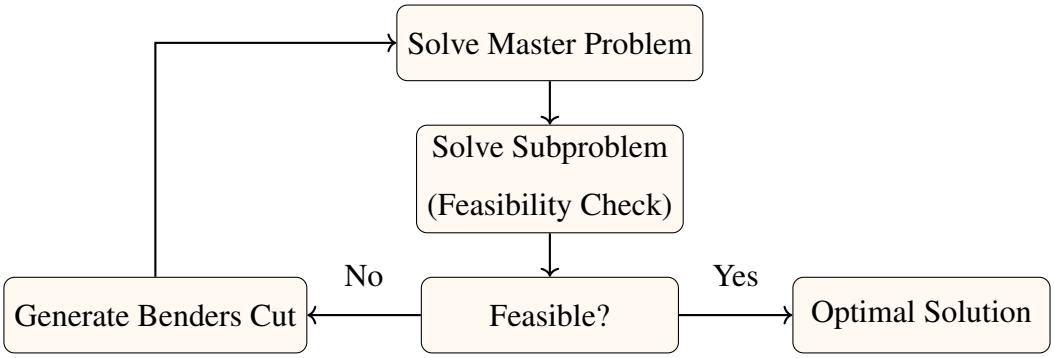

Figure 3.3: Benders Decomposition framework. The master problem proposes assignments, and the subproblem verifies feasibility, generating cuts when necessary.

3.5 LLM-Augmented Optimization Framework

While classical optimization techniques provide strong theoretical guarantees, they often rely on handcrafted heuristics or computationally expensive subroutines to explore large solution spaces. In this work, these methods are enhanced by integrating Large Language Models (LLMs) directly within the optimization loop, enabling adaptive rea-

soning, heuristic generation, and structured feedback.

The proposed framework combines Optimization by Prompting (OPRO) with ReAct-style reasoning. Instead of replacing optimization algorithms entirely, the LLM is embedded within key components of each method, improving efficiency while preserving correctness through validation.

3.5.1 LLM-Augmented Column Generation

In the classical Column Generation approach, the pricing subproblem is solved using exact combinatorial methods. In the proposed framework, this step is augmented using an LLM-based heuristic.

Given the dual prices from the master problem, the LLM generates candidate schedules that are likely to have negative reduced cost. These schedules are then validated using an exact pricing procedure before being added to the master problem. This ensures correctness while reducing computational effort.

The process is iterative, and feedback from previous iterations is used to refine the LLM's output.

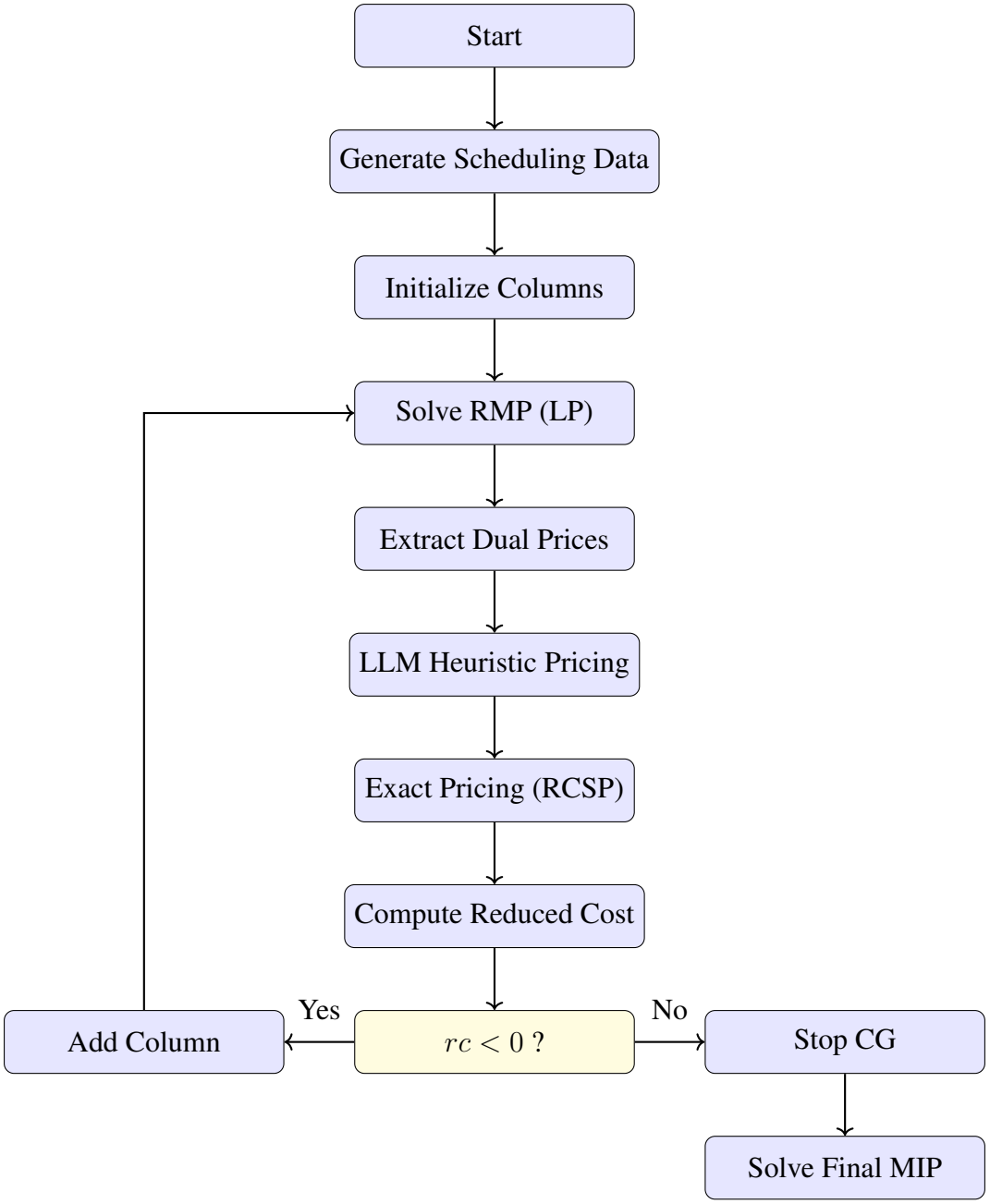

Figure 3.4: LLM-augmented Column Generation. The LLM proposes candidate columns which are validated using exact pricing.

3.5.2 LLM-Augmented Cutting Plane Method

In the Cutting Plane approach, identifying violated constraints is often computationally expensive. The LLM is used to analyze fractional solutions and suggest violated subsets.

These suggested cuts are validated using a fallback mechanism before being added to the model. This improves efficiency while maintaining correctness. The iterative

feedback allows the LLM to refine its ability to detect meaningful violations.

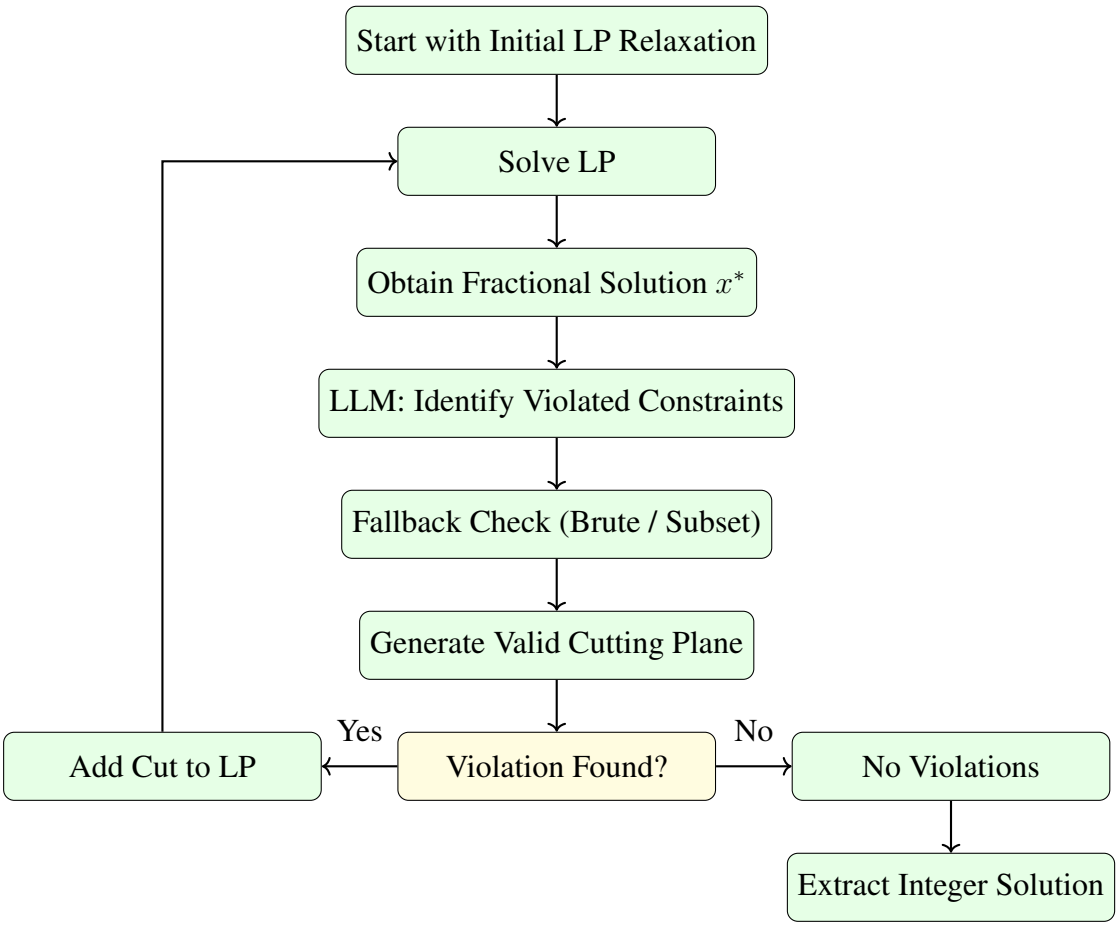

Figure 3.5: LLM-assisted Cutting Plane method. The LLM identifies candidate cuts which are validated before inclusion.

3.5.3 LLM-Augmented Benders Decomposition

In Benders Decomposition, the subproblem checks feasibility and generates cuts. The LLM assists in identifying infeasible patterns in schedules and proposing conflict-based cuts.

Instead of exhaustive enumeration, the LLM narrows the search space by suggesting likely violations. These are then validated before adding them to the master problem.

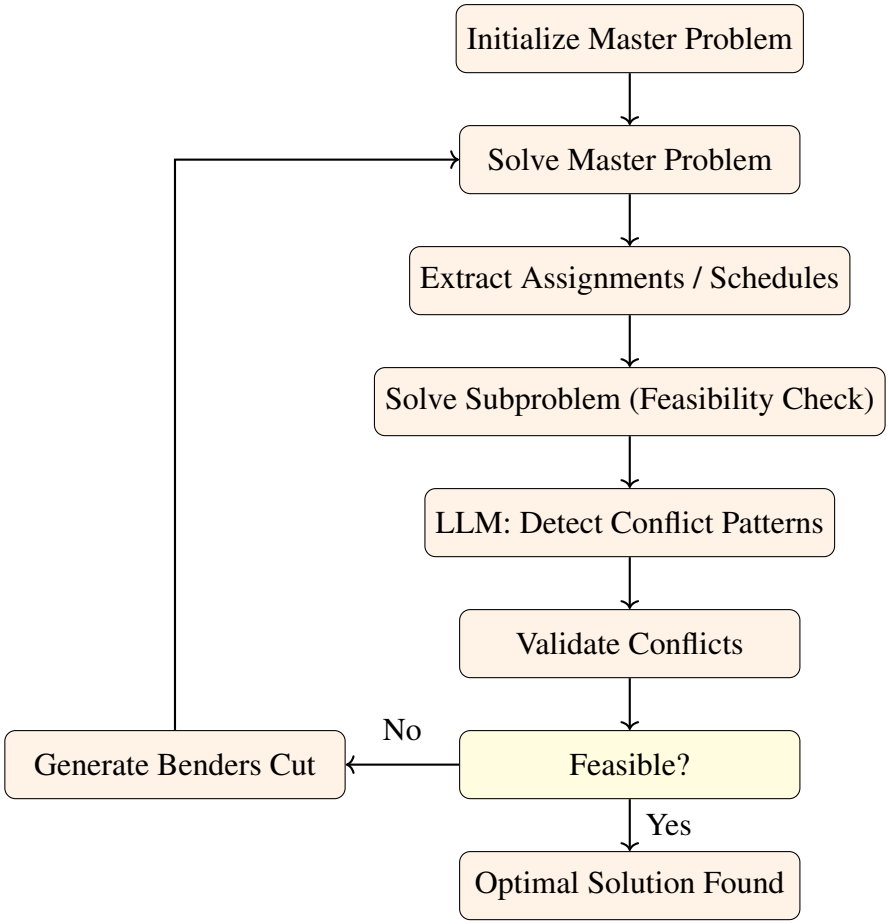

Figure 3.6: LLM-assisted Benders Decomposition. The LLM identifies infeasibilities which are converted into cuts.

3.5.4 Role of ReAct and Feedback Loop

A key component of the framework is the integration of ReAct-style reasoning, where the model alternates between reasoning and action steps. At each iteration, the model generates candidate outputs, receives structured feedback based on feasibility and objective value, and refines its responses accordingly.

This feedback-driven loop improves stability and reduces randomness. Compared to standard OPRO, the integration of reasoning enables more consistent convergence and better handling of constraints.

Overall, the proposed framework transforms classical optimization pipelines into adaptive systems capable of leveraging reasoning to guide search more effectively.

3.6 Experimental Setup

This section describes the experimental environment, data generation process, model configurations, and evaluation methodology used to assess the performance of classical optimization methods and the proposed OPRO + ReAct framework.

3.6.1 Problem Instance Generation

The hospital scheduling problem instances were synthetically generated to simulate realistic operational constraints. Each instance consists of a set of patients, where each patient is associated with:

- A time window $[s_i, e_i]$
- A required department
- A service duration
- A deadline constraint

Transition times between departments were incorporated to model real-world movement delays, such as:

$$\text{ICU} \rightarrow \text{Surgery} = 10 \text{ minutes}, \quad \text{Surgery} \rightarrow \text{Ward} = 5 \text{ minutes}$$

Additionally, operational constraints were enforced:

$$\text{Total duty time per doctor} \leq 8 \text{ hours}, \quad \text{Minimum rest time} \geq 1 \text{ hour}$$

These constraints ensure that generated schedules reflect realistic feasibility conditions.

3.6.2 Baseline Methods

Three classical optimization approaches were implemented:

- Column Generation (CG) with Restricted Master Problem and pricing subproblem

- Cutting Plane method with iterative constraint refinement
- Benders Decomposition separating assignment and feasibility

All baseline methods were implemented using standard optimization libraries and solved using linear programming (LP) and mixed-integer programming (MIP) formulations.

3.6.3 OPRO + ReAct Implementation

The proposed framework integrates Optimization by Prompting (OPRO) with ReAct-based reasoning. The process follows an iterative loop:

1. Generate candidate schedules using LLM-generated code
2. Execute the code on the generated problem instance
3. Evaluate feasibility and objective value
4. Provide structured feedback to the model
5. Refine the solution through subsequent iterations

Unlike standard OPRO, the ReAct component enables the model to:

- Debug infeasible schedules
- Identify constraint violations
- Suggest corrective structural changes

3.6.4 Model Selection

Based on the experimental results presented in Chapters 1 and 2, it was observed that GPT-OSS-120B consistently outperformed smaller models in terms of solution quality, feasibility, and convergence stability. In particular, it demonstrated superior reasoning capability and significantly fewer invalid or infeasible outputs across a wide range of combinatorial optimization problems.

Given these observations, all experiments in this chapter are conducted using GPT-OSS-120B. This allows us to focus on evaluating the effectiveness of the proposed

OPRO + ReAct framework in complex scheduling settings without variability introduced by weaker models.

This design choice ensures that the performance differences observed in this chapter are primarily due to the optimization framework rather than model limitations.

3.6.5 Evaluation Metrics

The performance of each method was evaluated based on:

- **Solution Quality:** Number of doctors used (minimization objective)
- **Feasibility:** Satisfaction of all constraints
- **Convergence Speed:** Number of iterations required
- **Computation Time:** Time taken to reach final solution

3.6.6 Experimental Protocol

For each method and model:

- The same problem instance was used
- Execution was repeated until convergence
- Both intermediate and final solutions were recorded
- Iteration-wise improvements were tracked

Additionally, for the OPRO + ReAct framework, both the generated code and execution outputs were logged to analyze the reasoning process.

3.6.7 Comparison Strategy

To ensure a meaningful comparison:

- Baseline methods were evaluated using exact solvers
- LLM-based methods were evaluated based on generated executable solutions
- Performance differences were analyzed in terms of:
  - Time to convergence

- Stability across iterations
- Ability to handle constraints

This setup allows for a direct comparison between traditional optimization techniques and LLM-driven approaches under identical conditions.

3.7 Results

This section presents the performance of the proposed framework on the Hospital Scheduling problem. The evaluation focuses on solution quality, convergence behavior, and computational efficiency across different problem sizes.

3.7.1 Column Generation Results

This section presents the results for the Column Generation method. Each experiment is shown using a single figure that includes both the convergence behavior and the final doctor schedule.

Baseline Performance

The classical Column Generation approach was evaluated on a 24-patient instance.

Patients	Doctors Required	Time Taken (s)
24	17	123.49

Table 3.1: Baseline Column Generation performance

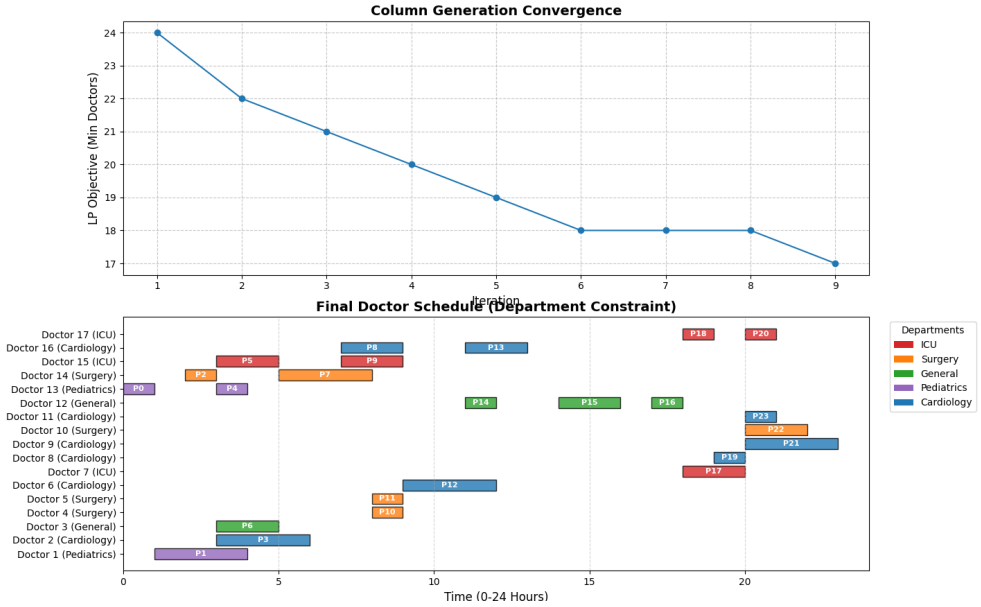

Figure 3.7: Baseline Column Generation: convergence and final schedule (24 patients)

The baseline method produces a valid schedule with 17 doctors, but requires a large computation time. The convergence is slow due to repeated exact pricing steps, making the approach inefficient for larger instances.

OPRO + ReAct Enhanced Performance

The enhanced method integrates LLM-based heuristic pricing, significantly improving efficiency while maintaining solution quality.

Patients	Doctors Required	Time Taken (s)
24	17	22.69
50	29	54.91
100	55	147.59

Table 3.2: OPRO + ReAct Column Generation performance

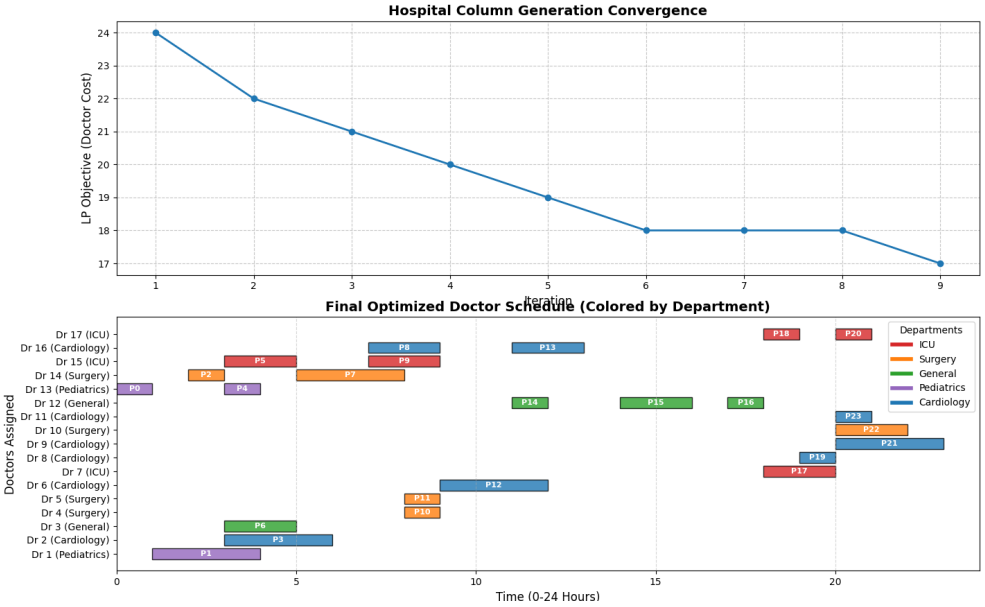

Figure 3.8: OPRO + ReAct Column Generation for 24 patients

The upper part of the figure shows the convergence of the LP objective, where the number of doctors decreases steadily from an initial estimate to the optimal value of 17. The lower part shows the final schedule, where all patients are assigned without violating constraints.

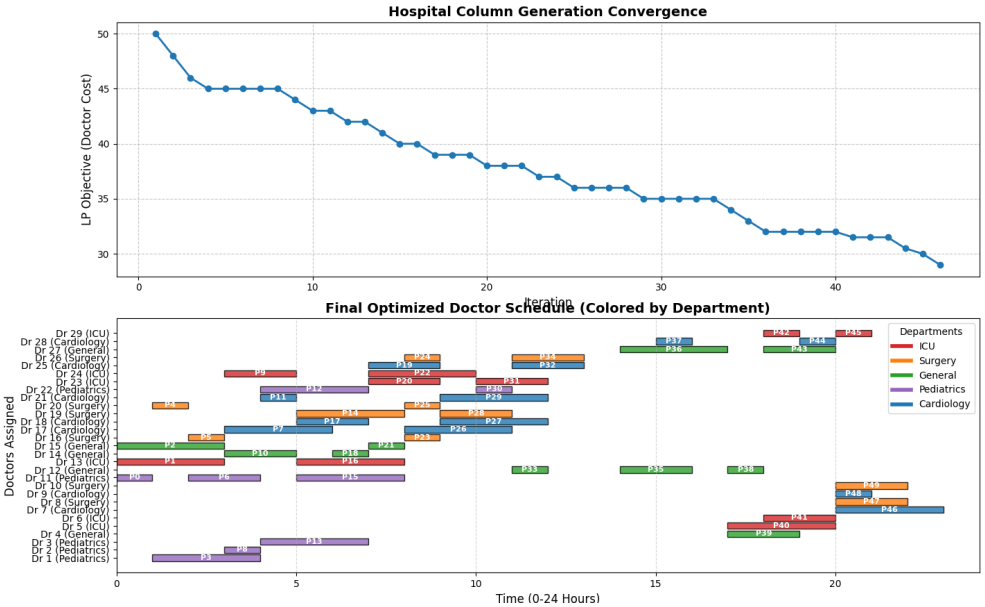

Figure 3.9: OPRO + ReAct Column Generation for 50 patients

For 50 patients, the convergence remains smooth and stable. The schedule shows structured grouping of patients by department, indicating that the model effectively captures constraint relationships.

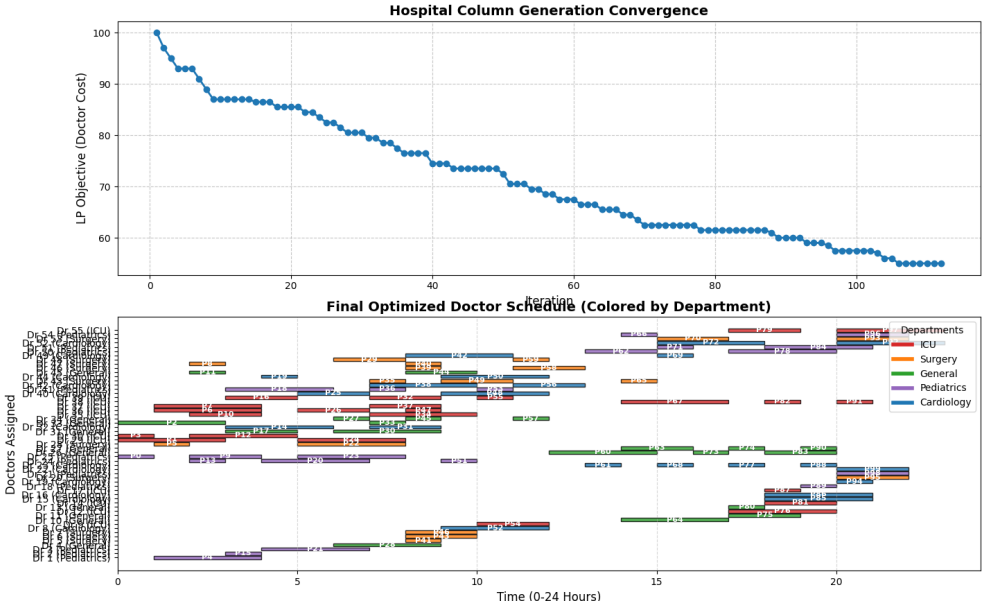

Figure 3.10: OPRO + ReAct Column Generation for 100 patients

For larger instances, the number of iterations increases, but convergence remains monotonic. The resulting schedule is feasible and well-organized, demonstrating scalability.

Analysis

The comparison between the baseline and enhanced approaches shows a significant improvement in computational efficiency. For the 24-patient instance, both methods achieve the same optimal solution of 17 doctors, but the computation time is reduced from 123.49 seconds to 22.69 seconds.

More importantly, the baseline method exhibits a rapid increase in computation time as the problem size grows. This behavior is consistent with observations from other methods evaluated in this study. For instance, in the Cutting Plane approach, the baseline runtime for a 24-patient instance reached 922.07 seconds, making it computationally impractical even for moderately sized problems. Similarly, baseline implementations of Benders Decomposition required significantly higher runtimes even for smaller instances.

These results indicate that although decomposition-based methods improve over direct formulations, their baseline implementations still face scalability challenges due to expensive subproblem computations and iterative refinement steps. In contrast, the

enhanced approach maintains controlled growth in runtime and remains efficient for larger instances such as 50 and 100 patients.

The integration of LLM-based heuristic pricing allows the algorithm to identify promising columns early, reducing the reliance on expensive exact pricing steps. This results in faster convergence, improved stability, and practical scalability for larger problem instances.

3.7.2 Cutting Plane Results

This section presents the performance of the Cutting Plane method for the hospital scheduling problem. We compare the baseline implementation with the OPRO + ReAct enhanced approach in terms of both solution quality and computation time.

Baseline Performance

The classical Cutting Plane method was evaluated on a 24-patient instance.

Patients	Doctors Required	Time Taken (s)
24	17	922.07

Table 3.3: Baseline Cutting Plane performance

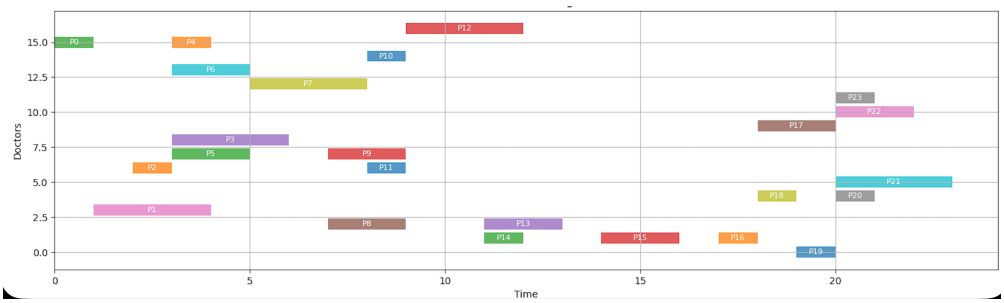

Figure 3.11: Baseline Cutting Plane schedule (24 patients)

The baseline method produces a feasible schedule satisfying all constraints. However, the computation time is extremely high due to exhaustive enumeration of violated subsets, which grows combinatorially with the number of patients.

OPRO + ReAct Enhanced Performance

The enhanced method integrates LLM-guided reasoning to identify violated constraints more efficiently, significantly reducing computation time while maintaining solution quality.

Patients	Doctors Required	Time Taken (s)
24	17	45.68
50	29	81.34
100	55	142.85

Table 3.4: OPRO + ReAct Cutting Plane performance

Comparison for 24 Patients

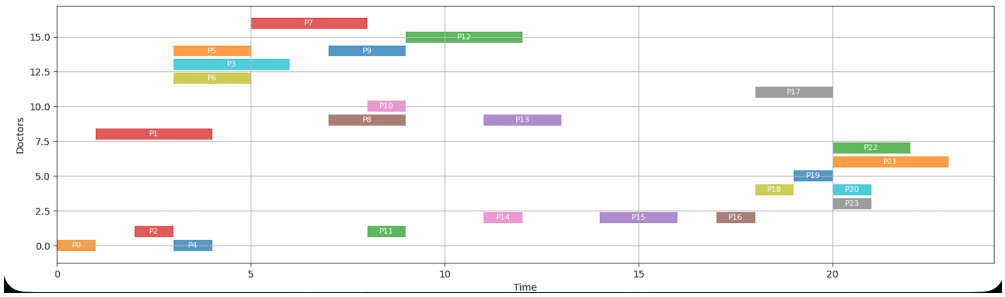

Figure 3.12: OPRO + ReAct Cutting Plane schedule (24 patients)

Both methods achieve the same optimal solution requiring 17 doctors. However, the enhanced method reduces computation time from 922.07 seconds to 45.68 seconds, representing a substantial improvement.

Scalability Results

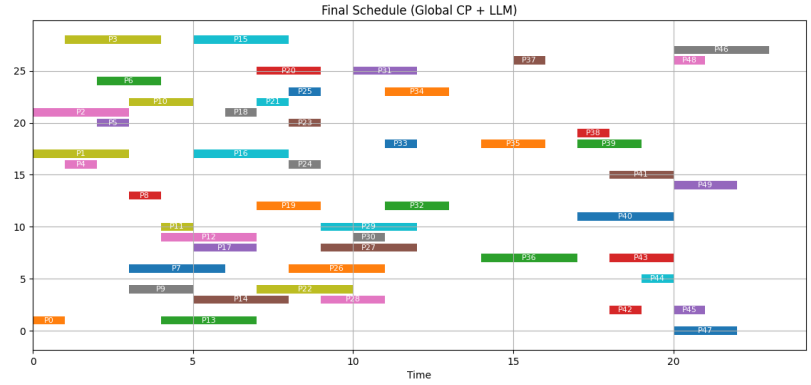

Figure 3.13: OPRO + ReAct Cutting Plane schedule (50 patients)

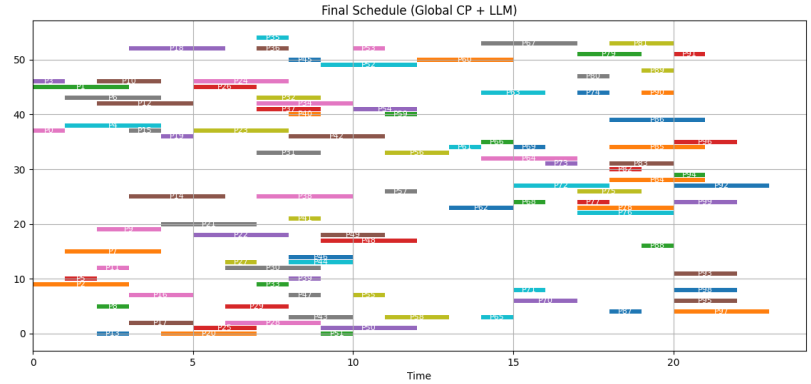

Figure 3.14: OPRO + ReAct Cutting Plane schedule (100 patients)

The baseline method becomes computationally impractical beyond small instances, whereas the enhanced method continues to scale to larger problem sizes such as 50 and 100 patients.

Analysis

The integration of OPRO + ReAct leads to a significant improvement in computational efficiency. While both methods produce identical optimal solutions, the enhanced method reduces the reliance on exhaustive enumeration by guiding the search toward promising constraint violations.

Although the improvement is less extreme compared to earlier observations, the runtime reduction remains substantial, with approximately a 20× speedup for the 24-

patient instance. More importantly, the enhanced method demonstrates stable scaling behavior as the problem size increases.

The resulting schedules remain feasible and well-structured, confirming that the improvement in efficiency does not compromise solution quality. This makes the enhanced Cutting Plane approach practical for real-world scheduling scenarios.

3.7.3 Benders Decomposition Results

This section presents the performance of Benders Decomposition for the hospital scheduling problem. We compare the baseline implementation with the OPRO + ReAct enhanced approach in terms of convergence behavior and computational efficiency.

Baseline Performance

The classical Benders Decomposition method was evaluated on an 18-patient instance.

Patients	Doctors Required	Time Taken (s)
18	13	65.61

Table 3.5: Baseline Benders Decomposition performance

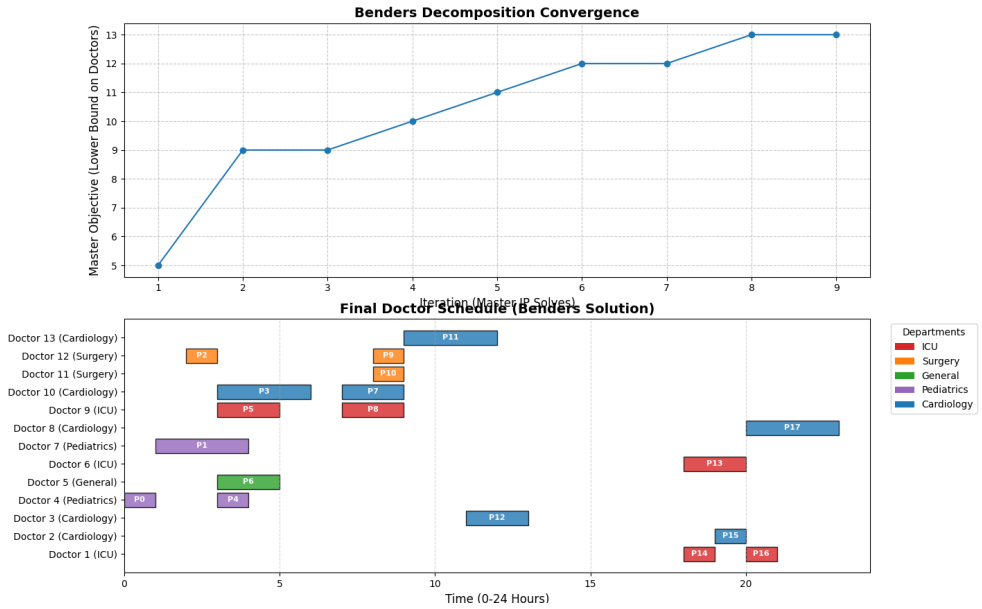

Figure 3.15: Baseline Benders Decomposition: convergence and final schedule (18 patients)

The baseline method converges to a feasible and optimal solution. However, the iterative generation of feasibility cuts leads to higher computational overhead, increasing the total runtime.

OPRO + ReAct Enhanced Performance

The enhanced approach integrates LLM-guided reasoning to generate more effective cuts, reducing unnecessary iterations and improving convergence efficiency.

Patients	Doctors Required	Time Taken (s)
18	13	13.82

Table 3.6: OPRO + ReAct Benders Decomposition performance

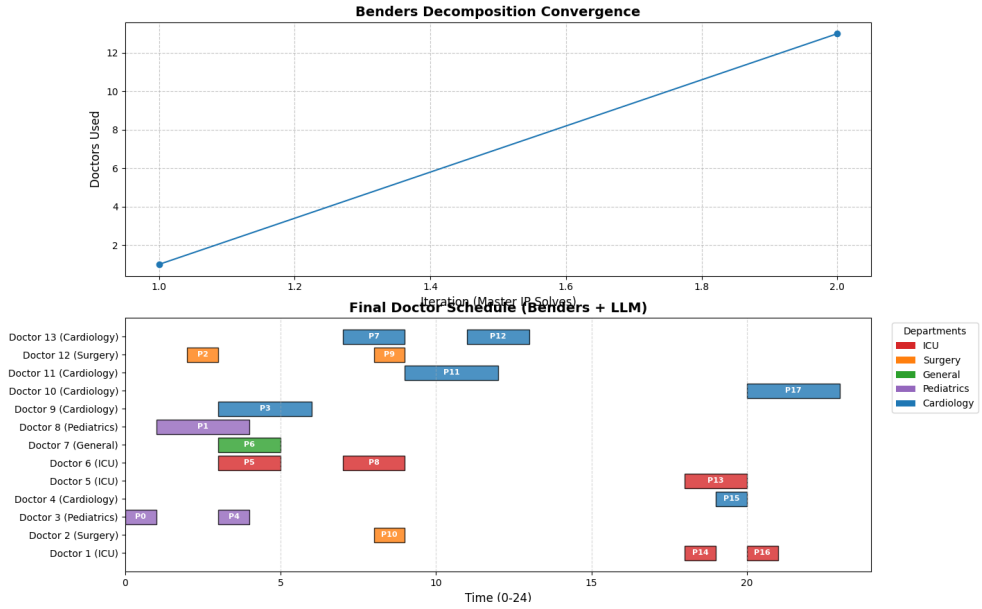

Figure 3.16: OPRO + ReAct Benders Decomposition: convergence and final schedule (18 patients)

The enhanced method achieves the same optimal solution using significantly less computation time.

Analysis

The results demonstrate that integrating structured reasoning into Benders Decomposition improves computational efficiency without affecting solution quality. The LLM

assists in identifying effective feasibility cuts, reducing redundant exploration of infeasible schedules.

However, unlike Column Generation and Cutting Plane methods, Benders Decomposition still exhibits rapid growth in computation time as the problem size increases, even after incorporating OPRO + ReAct. Due to this behavior, experiments for larger instances were not computationally feasible within a reasonable time limit.

As a result, the comparison is limited to smaller instances (18 patients), where both baseline and enhanced methods could be evaluated reliably. The consistency of results for this instance confirms the correctness of the enhanced approach, although scalability remains a limitation for Benders Decomposition in this setting.

While the improvement in runtime is significant, the overall impact of LLM integration is comparatively smaller, as the method primarily depends on master–subproblem interactions rather than large-scale combinatorial search.

3.8 Comparative Evaluation

This chapter evaluated three optimization approaches for hospital scheduling: Column Generation, Cutting Plane, and Benders Decomposition. Each method was tested in both its baseline form and with the OPRO + ReAct enhancement.

Across all methods, the enhanced framework consistently produced solutions of identical quality while significantly reducing computation time. However, the degree of improvement varied across methods.

Column Generation demonstrated the strongest overall performance. The integration of OPRO + ReAct enabled faster convergence by guiding the pricing step toward high-quality columns early in the optimization process. This resulted in substantial reductions in runtime and allowed the method to scale efficiently to larger instances such as 50 and 100 patients, while maintaining stable convergence behavior.

The Cutting Plane method also showed significant improvement. The enhanced framework reduced the need for exhaustive enumeration of violated subsets by identifying promising cuts through structured reasoning. While the improvement was sub-

stantial, the method still exhibited higher computational cost compared to Column Generation, particularly as the problem size increased.

Benders Decomposition benefited from the enhancement as well, although to a lesser extent. The reduction in runtime was noticeable but smaller compared to the other methods. This is primarily because Benders decomposition relies more on master-subproblem interactions rather than large-scale combinatorial search, limiting the impact of LLM-guided reasoning.

Overall, the results indicate that while the OPRO + ReAct framework improves all optimization methods, Column Generation emerges as the most effective approach in terms of scalability, stability, and computational efficiency.

3.9 Discussion

The experimental results demonstrate that incorporating structured reasoning into optimization algorithms leads to consistent and meaningful performance improvements. The ReAct mechanism enables iterative refinement of candidate solutions, allowing the model to generate, evaluate, and correct decisions within the optimization loop.

A key observation is the shift from unguided search to informed decision-making. In the baseline setting, optimization methods rely heavily on exhaustive exploration or heuristic ordering, which becomes inefficient as the problem size grows. With OPRO + ReAct, the model directs the search toward promising regions of the solution space, reducing unnecessary computations.

The effectiveness of this integration, however, depends on the structure of the underlying optimization method. In Column Generation, where the pricing problem involves searching over a large combinatorial space, LLM guidance significantly accelerates convergence. This explains why Column Generation achieves the best performance among all methods.

In contrast, the Cutting Plane method benefits from improved identification of violated constraints, but still requires repeated LP re-optimization, which limits overall efficiency gains. Similarly, Benders Decomposition sees moderate improvement, as its bottleneck lies in solving structured subproblems rather than exploring large candidate

spaces.

Another important insight is the improvement in stability and consistency. The enhanced framework reduces variability in solution quality and avoids erratic behavior observed in purely heuristic approaches. This makes the optimization process more reliable and predictable.

Overall, the results highlight that LLM-based reasoning is most effective when integrated into components of optimization algorithms that involve combinatorial search or candidate generation, rather than deterministic subproblem solving.

3.10 Chapter Summary

In this chapter, we extended the OPRO framework using ReAct and applied it to the hospital scheduling problem. The enhanced approach was evaluated across three optimization methods: Column Generation, Cutting Plane, and Benders Decomposition.

The results show that integrating structured reasoning into the optimization loop significantly improves computational efficiency while preserving solution quality. Among the methods studied, Column Generation achieved the best overall performance, demonstrating superior scalability, faster convergence, and greater stability.

The Cutting Plane method also benefited from the enhancement, particularly in reducing the cost of constraint generation, while Benders Decomposition showed moderate improvements.

These findings suggest that the combination of classical optimization techniques with LLM-based reasoning is a promising direction for solving complex scheduling problems. In the next chapter, this framework is extended to the flight crew assignment problem, which introduces additional constraints and operational complexity, providing a more challenging evaluation setting.

CHAPTER 4

Flight Crew Assignment

4.1 Introduction

Flight crew assignment is a classical and highly constrained optimization problem in airline operations. Unlike simplified scheduling problems, crew assignment must simultaneously account for temporal feasibility, spatial continuity, regulatory duty limits, and operational constraints such as rest requirements and airport transfers. These constraints make the problem inherently combinatorial and significantly more complex than standard scheduling formulations.

The motivation for studying this problem lies in its practical importance and computational difficulty. Efficient crew assignment directly impacts operational costs, crew utilization, and service reliability. Traditional approaches such as Column Generation and Benders Decomposition have been widely used in practice, but they often require carefully designed heuristics and suffer from scalability challenges when the number of flights increases.

Building on the findings from Chapter 3, this chapter evaluates whether Large Language Models (LLMs), when integrated using the OPRO + ReAct framework, can improve the performance of classical optimization methods in this more complex setting. In particular, we investigate whether structured reasoning can reduce computational effort while maintaining solution quality under realistic constraints.

4.2 Problem Formulation

A flight operator manages a set of daily flights. Each flight requires exactly one crew. A crew can operate multiple flights in sequence, provided that the following operational and regulatory constraints are satisfied:

1. Flights are time-compatible, i.e., the next flight must start after the previous one ends including turnaround time.

2. Location continuity is maintained, meaning the destination of the previous flight must match the origin of the next flight, or be reachable via repositioning within the available time.
3. The total duty time of a crew does not exceed legal limits (typically between 8 to 10 hours; in our experiments fixed at 10 hours).
4. Minimum rest or turnaround time between consecutive flights is satisfied (e.g., 30–60 minutes, applied before and after flights).
5. Airport transfer time is respected, accounting for additional travel time required when flights operate from different terminals or airports.
6. Each flight must be operated before its scheduled departure deadline.
7. Each crew must start from a valid base location or a location that is reachable within the allowed time.

The objective is to assign crews to flights such that all flights are covered exactly once, all operational and legal constraints are satisfied, and the total number of crews used is minimized.

This formulation results in a highly constrained combinatorial optimization problem, where feasible solutions correspond to valid sequences of flights (crew routes) that satisfy all the above constraints simultaneously.

4.3 Mathematical Formulation

Let F denote the set of flights and K denote the set of feasible crew routes. Each route represents a sequence of flights that can be operated by a single crew while satisfying all constraints.

Define binary decision variables:

$$x_k = \begin{cases} 1 & \text{if route } k \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

Each route k covers a subset of flights. Let $a_{fk} = 1$ if flight f is included in route k, and 0 otherwise.

The optimization problem can be written as:

$$\min \sum_{k \in K} x_k$$

subject to:

$$\sum_{k \in K} a_{fk} x_k = 1 \quad \forall f \in F$$

$$x_k \in \{0, 1\} \quad \forall k \in K$$

The feasibility of each route k is determined by:

- Time compatibility between consecutive flights
- Location continuity constraints
- Duty time limits
- Minimum rest requirements
- Airport transfer feasibility
- Base location constraints

Since the number of feasible routes grows exponentially with the number of flights, direct enumeration is not practical. This motivates the use of decomposition techniques such as Column Generation, Cutting Plane, and Benders Decomposition, which are enhanced in this work using LLM-guided reasoning.

4.4 Optimization Frameworks

The optimization methods used in this chapter—Column Generation, Cutting Plane, and Benders Decomposition—are based on the same theoretical foundations described in Chapter 3. Therefore, we avoid repeating the full derivations and instead focus on how these methods are adapted to the flight crew assignment problem and how the OPRO + ReAct framework enhances each approach.

Compared to the hospital scheduling problem, the flight crew assignment setting introduces additional constraints such as location continuity, airport transfer feasibility,

and base constraints. These significantly increase the complexity of generating feasible schedules, making intelligent guidance during optimization more critical.

4.4.1 Column Generation Framework

Column Generation constructs feasible crew routes iteratively. Each column represents a valid sequence of flights that satisfies time, location, and duty constraints.

The Restricted Master Problem (RMP) selects a subset of these routes to cover all flights. The pricing subproblem identifies new routes with negative reduced cost, typically formulated as a resource-constrained shortest path problem (RCSP).

The OPRO + ReAct framework enhances this process by allowing the LLM to propose candidate routes based on dual prices. These routes are validated and compared with exact dynamic programming solutions before inclusion.

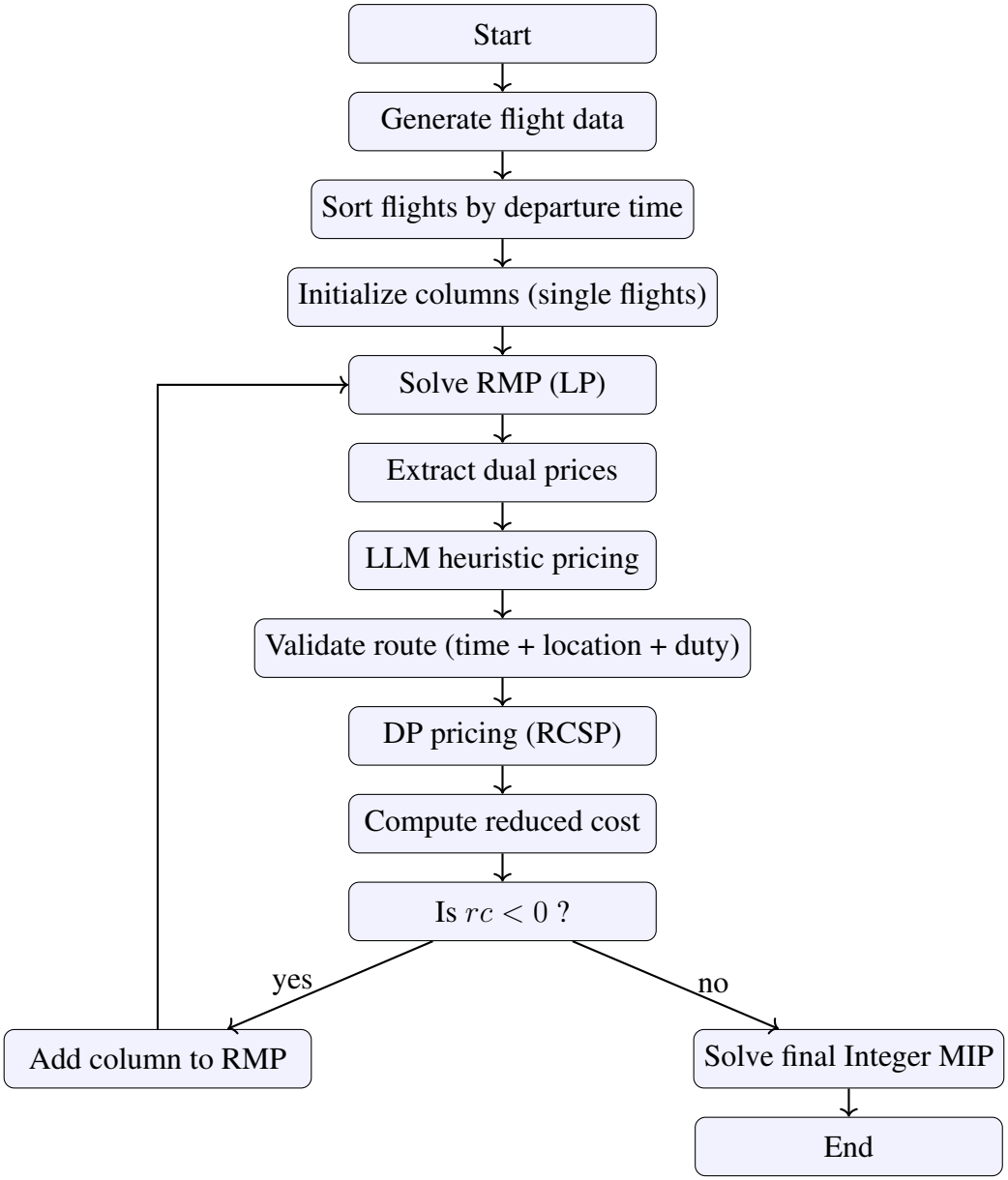

Figure 4.1: Column Generation with LLM-guided iterative pricing

4.4.2 Cutting Plane Framework

The Cutting Plane method iteratively refines a relaxed solution by adding violated constraints (cuts). Each iteration solves an LP relaxation, identifies infeasible structures, and adds constraints to eliminate them.

In this setting, infeasibilities arise from violations of time compatibility, location continuity, or duty limits within a constructed crew schedule.

The LLM is used to identify violated subsets more efficiently by analyzing fractional solutions and proposing candidate conflicts. These are then validated and added as cuts

to the model.

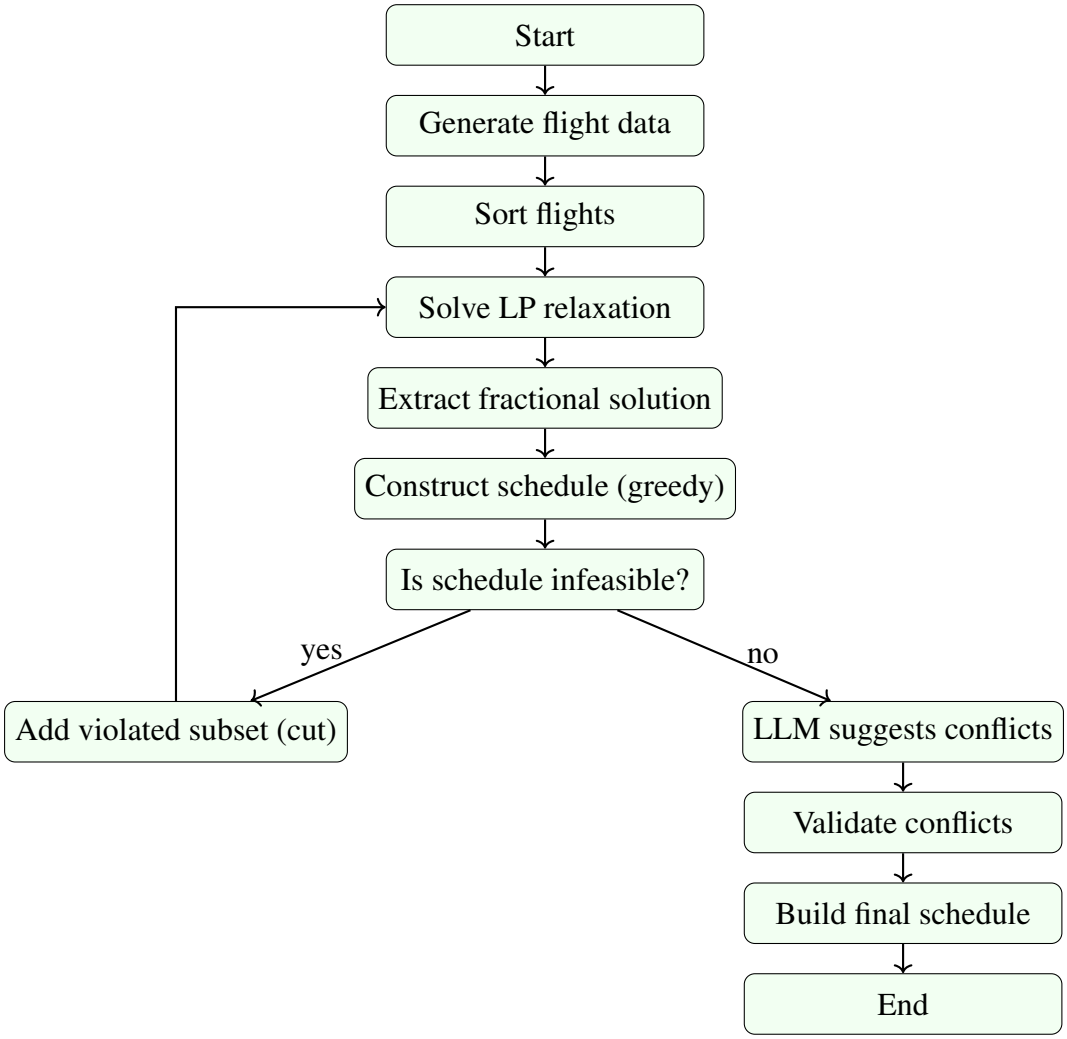

Figure 4.2: Cutting Plane with iterative cut generation and LLM guidance

4.4.3 Benders Decomposition Framework

Benders Decomposition separates the problem into a master assignment problem and a feasibility subproblem. The master problem proposes assignments of flights to crews, and the subproblem verifies whether these assignments satisfy all operational constraints.

If infeasibilities are detected, Benders cuts are generated and added to the master problem, and the process is repeated.

The LLM assists in identifying likely conflicts within schedules, reducing the burden of exhaustive feasibility checks.

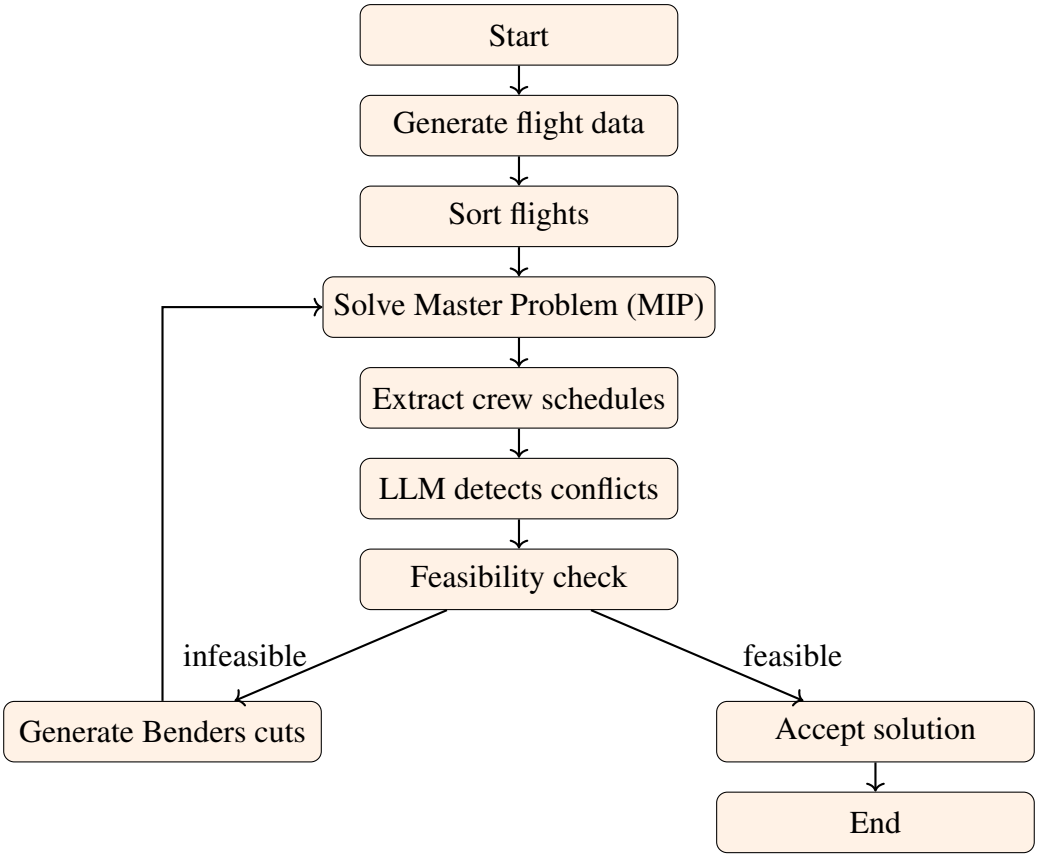

Figure 4.3: Benders Decomposition with iterative feasibility feedback

4.4.4 Role of OPRO + ReAct

Across all three methods, the OPRO + ReAct framework introduces a structured reasoning loop into the optimization process. Instead of relying purely on deterministic heuristics or exhaustive search, the LLM proposes candidate solutions, identifies conflicts, and guides the optimization trajectory.

Crucially, all LLM-generated outputs are validated using exact checks, ensuring correctness is never compromised. This hybrid design enables faster convergence while maintaining feasibility and solution quality, particularly in the presence of complex real-world constraints.

4.5 Results

This section presents the performance evaluation of the proposed framework for the flight crew assignment problem. The comparison includes both the baseline approach

and the enhanced OPRO + ReAct framework.

4.5.1 Column Generation Performance

We first analyze the results for the Column Generation method.

Baseline Method

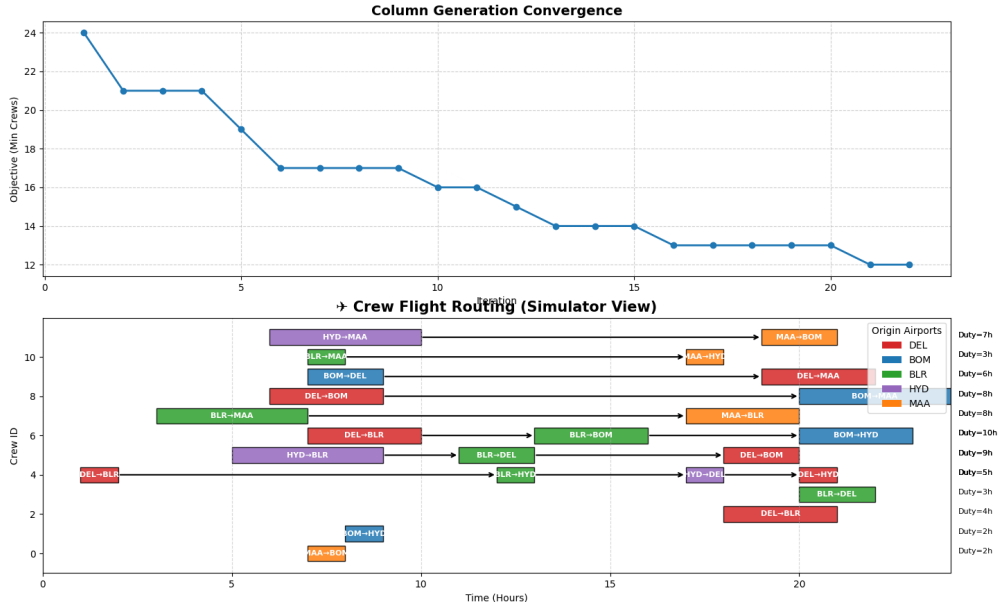

Figure 4.4: Baseline Column Generation: Crew routing for 24 flights

The baseline implementation requires 335.64 seconds to solve the 24-flight instance, producing a solution with 12 crews. The computation is dominated by exhaustive pricing and lacks guided search, leading to slow convergence.

Proposed Method (OPRO + ReAct)

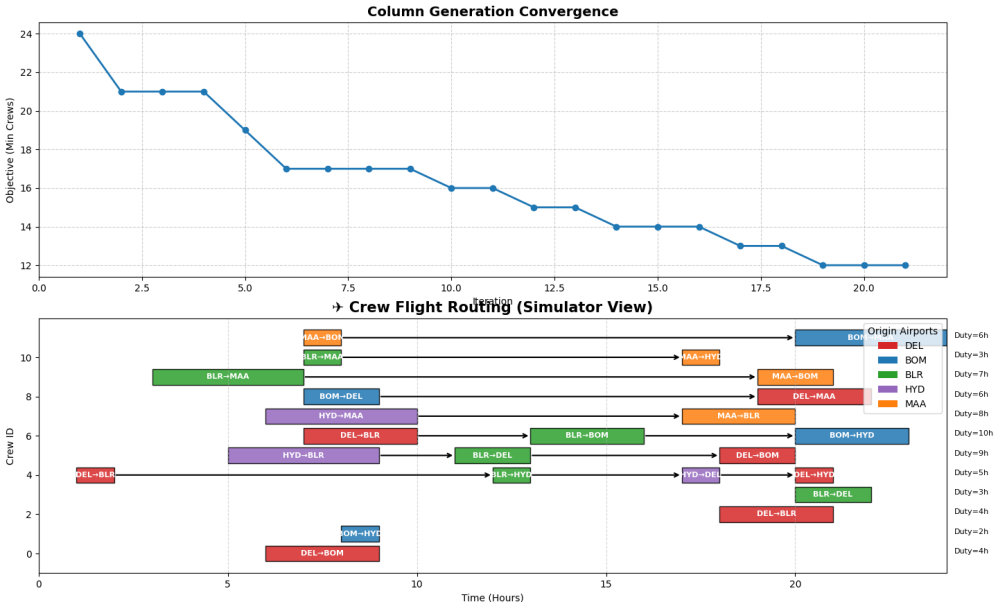

Figure 4.5: Proposed Method: Convergence and routing for 24 flights

The proposed method achieves the same solution (12 crews) in only 12.43 seconds, representing a significant improvement in computational efficiency.

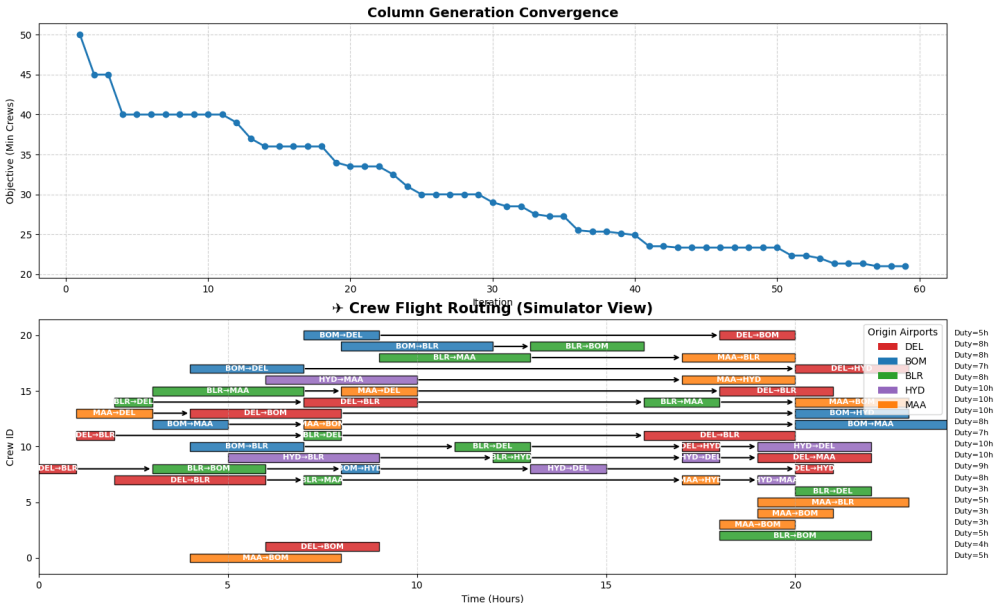

Figure 4.6: Proposed Method: Convergence and routing for 50 flights

For 50 flights, the algorithm produces a solution with 21 crews in 20.22 seconds. The convergence plot shows steady improvement in the objective value.

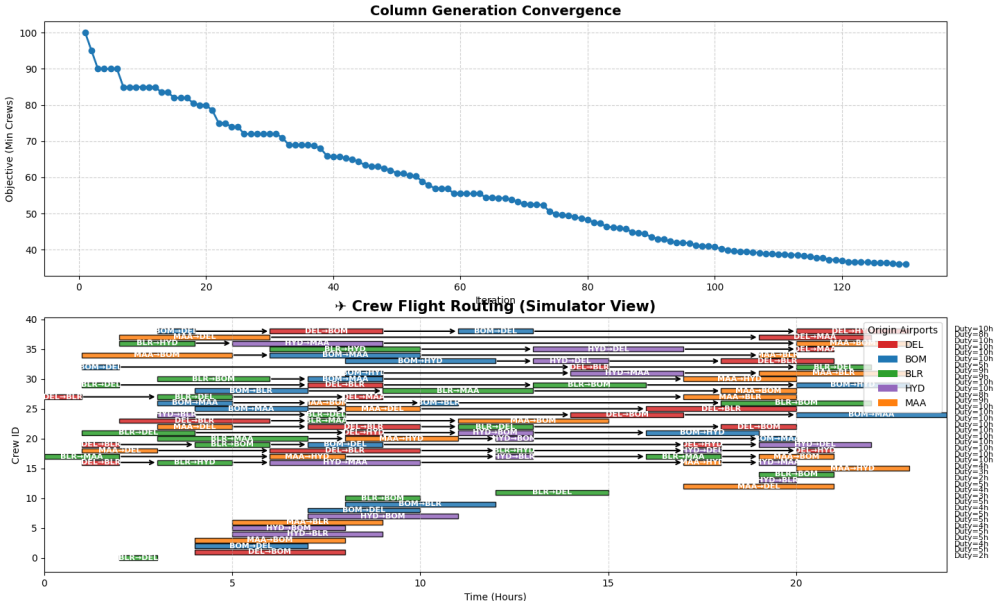

Figure 4.7: Proposed Method: Convergence and routing for 100 flights

For 100 flights, the method scales effectively, requiring 39 crews and 30.68 seconds. The convergence remains stable even for larger instances.

4.5.2 Comparison with Baseline

A direct comparison highlights the efficiency of the proposed framework.

Method	Flights	Crews	Time (seconds)
Baseline	24	12	335.64
Proposed	24	12	12.43
Proposed	50	21	20.22
Proposed	100	39	30.68

Table 4.1: Comparison of baseline and proposed methods

The proposed approach achieves identical solution quality while reducing computation time by more than an order of magnitude.

4.5.3 Convergence Behavior

The convergence plots show a monotonic decrease in the objective value as new columns are added. Early iterations contribute significantly to improvement, while later itera-

tions refine the solution gradually.

The LLM-guided pricing mechanism accelerates convergence by generating high-quality candidate routes early in the process, reducing the number of required iterations.

4.5.4 Scalability

The results demonstrate that the proposed framework scales efficiently with problem size. While the baseline approach exhibits rapid growth in computation time, the enhanced method maintains manageable runtimes even for larger instances.

4.5.5 Summary

Overall, the integration of OPRO + ReAct with Column Generation leads to substantial improvements in efficiency without compromising solution quality. The method provides a scalable and robust solution for real-world flight crew scheduling problems.

4.5.6 Cutting Plane Results

This section presents the performance of the Cutting Plane method for the flight crew scheduling problem. The baseline implementation is compared with the OPRO + ReAct enhanced approach in terms of solution quality, computational efficiency, and scalability.

Baseline Performance

The classical Cutting Plane method was evaluated on a 22-flight instance.

Flights	Crews Required	Time Taken (s)
22	11	59.17

Table 4.2: Baseline Cutting Plane performance

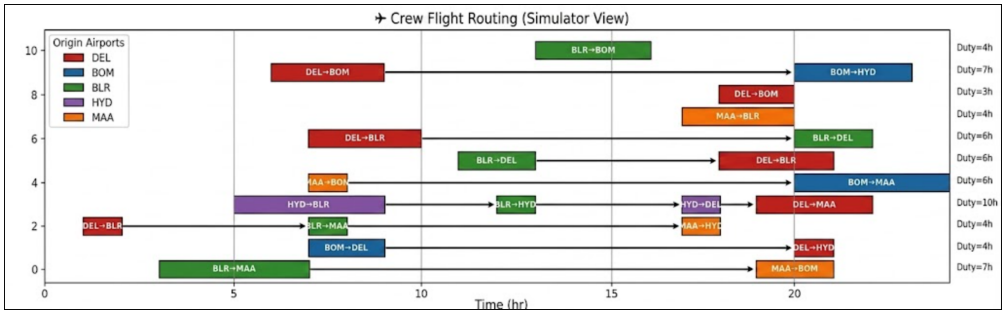

Figure 4.8: Baseline Cutting Plane schedule (22 flights)

The baseline method produces a feasible schedule satisfying all operational constraints, including time compatibility, location continuity, duty limits, and turnaround requirements. However, the runtime is relatively high due to repeated identification of violated subsets through exhaustive search.

OPRO + ReAct Enhanced Performance

The enhanced method integrates LLM-guided reasoning to identify violated constraints and generate feasible crew chains more efficiently.

Flights	Crews Required	Time Taken (s)
22	11	34.9
50	21	52.02
100	39	54.16

Table 4.3: OPRO + ReAct Cutting Plane performance

Comparison for 22 Flights

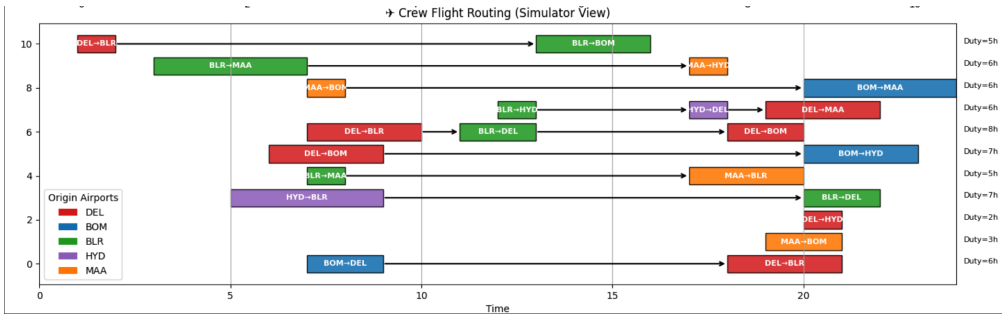

Figure 4.9: OPRO + ReAct Cutting Plane schedule (22 flights)

Both methods achieve identical solutions requiring 11 crews. However, the enhanced method reduces computation time from 59.17 seconds to 34.9 seconds, demonstrating a clear improvement in efficiency.

Scalability Results

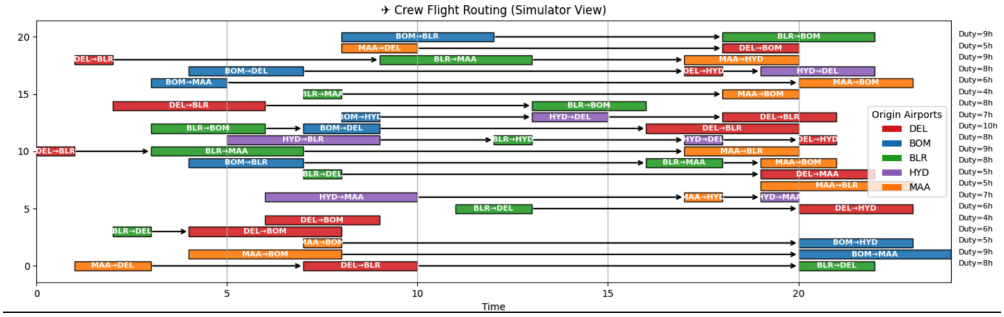

Figure 4.10: OPRO + ReAct Cutting Plane schedule (50 flights)

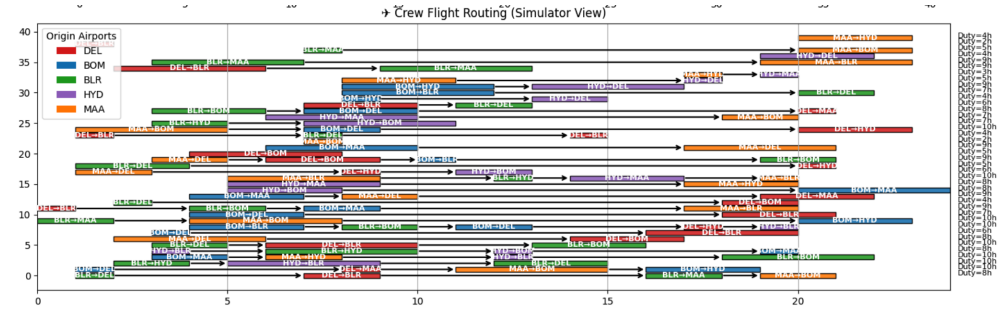

Figure 4.11: OPRO + ReAct Cutting Plane schedule (100 flights)

The enhanced method scales efficiently to larger problem sizes. Notably, the runtime increase from 50 to 100 flights is minimal, indicating that the LLM-guided approach effectively controls the growth of the search space.

Analysis

The integration of OPRO + ReAct improves the Cutting Plane method by reducing the cost of identifying violated constraints and guiding the construction of feasible crew schedules.

Unlike the hospital setting, where the improvement was several orders of magnitude, the gains in the flight problem are more moderate. This is because the flight

scheduling constraints (time, location, and duty) impose a more structured feasibility space, reducing the need for extensive combinatorial exploration.

However, the enhanced method still provides consistent runtime reductions and significantly better scalability. The ability to handle 50 and 100 flight instances efficiently highlights the robustness of the approach.

Overall, while Cutting Plane benefits from LLM-guided reasoning, it remains less efficient compared to Column Generation for this problem, primarily due to its iterative constraint generation framework.

4.5.7 Benders Decomposition Results

This section evaluates the performance of the Benders Decomposition method for the flight crew scheduling problem. Due to the computational complexity of the baseline approach, comparisons are performed across different instance sizes while verifying solution consistency.

Baseline Performance

The classical Benders Decomposition method was evaluated on a smaller instance due to computational limitations.

Flights	Crews Required	Time Taken (s)
12	8	240.36

Table 4.4: Baseline Benders performance

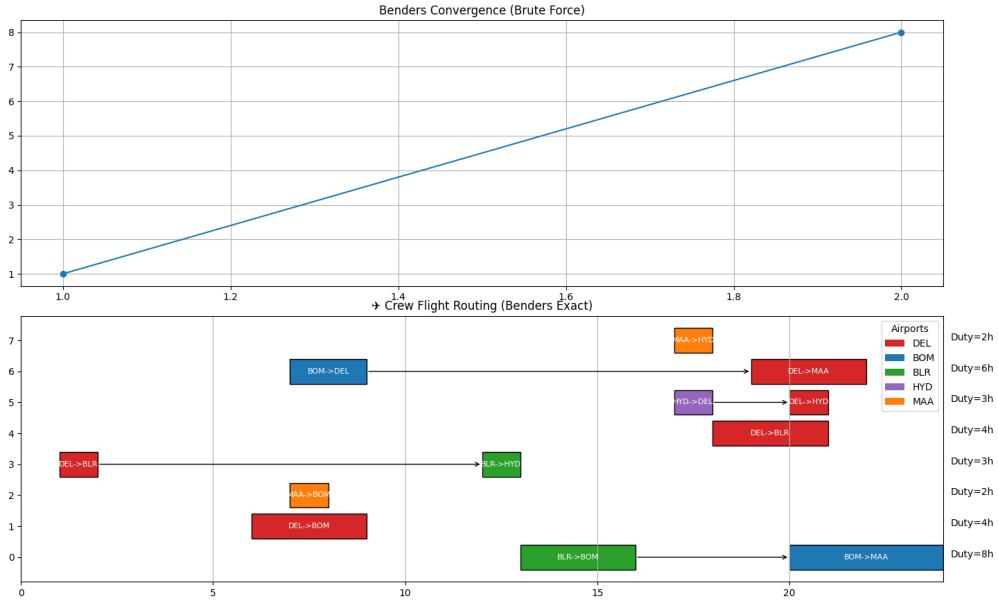

Figure 4.12: Baseline Benders schedule (12 flights)

The baseline method produces a valid and optimal schedule satisfying all operational constraints. However, the runtime is extremely high even for small instances. The repeated solution of the master problem and exhaustive feasibility checks in the subproblem lead to rapid growth in computation time.

OPRO + ReAct Enhanced Performance

The enhanced method integrates LLM-guided reasoning to accelerate conflict detection and reduce unnecessary feasibility checks.

Flights	Crews Required	Time Taken (s)
22	11	14.53

Table 4.5: OPRO + ReAct Benders performance

Comparison and Validation

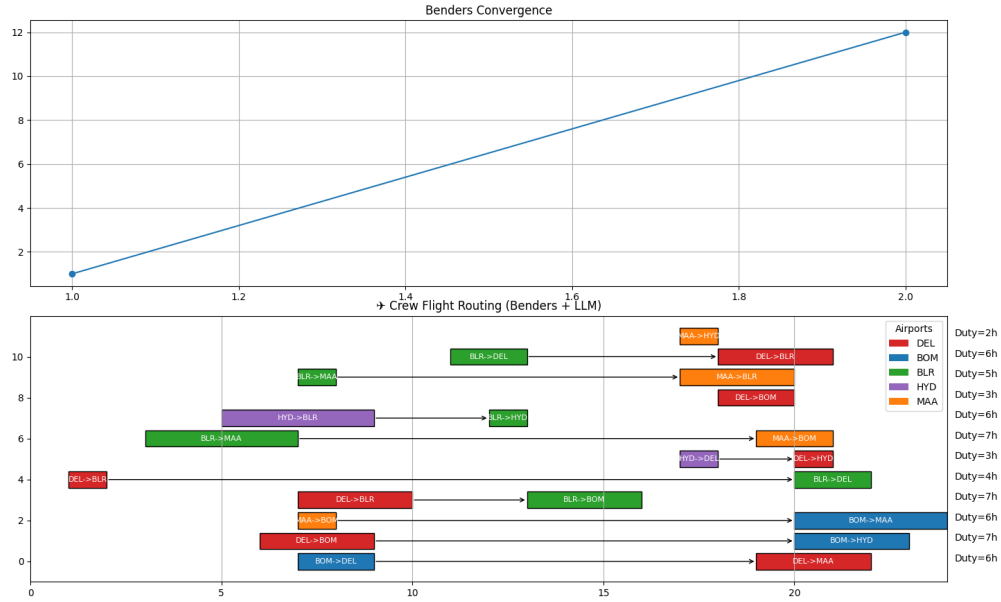

Figure 4.13: OPRO + ReAct Benders schedule (22 flights)

A direct comparison at identical scales is not feasible because the baseline method becomes computationally intractable beyond 12 flights. However, solution consistency was verified by comparing with results from other methods (Column Generation and Cutting Plane), confirming that 22 flights require 11 crews.

Analysis

The integration of OPRO + ReAct significantly improves the efficiency of Benders Decomposition. The primary bottleneck in the baseline approach lies in the subproblem, where feasibility is checked using exhaustive enumeration.

The enhanced method mitigates this issue by leveraging LLM-based reasoning to detect conflicts early in the process, which significantly reduces the total number of expensive feasibility checks required. Furthermore, this approach allows for the generation of more informative cuts, leading to a more efficient and targeted search for the optimal solution.

As a result, runtime is reduced from 240.36 seconds (for 12 flights) to 14.53 seconds (for 22 flights), while maintaining solution correctness.

Despite these improvements, Benders Decomposition remains less scalable com-

pared to Column Generation. This is because it still relies on iterative master-subproblem coordination, which introduces overhead even with improved cut generation.

Overall, the enhanced Benders approach demonstrates significant performance gains but is best suited for moderate-sized instances compared to the other methods.

4.6 Comparative Evaluation

This chapter evaluated three optimization approaches for the flight crew scheduling problem: Column Generation, Cutting Plane, and Benders Decomposition. Each method was analyzed in both its baseline form and with the OPRO + ReAct enhancement.

Among all methods, Column Generation demonstrated the best overall performance. It achieved the lowest computation time, stable convergence behavior, and strong scalability across all tested problem sizes. The integration of OPRO + ReAct further improved its efficiency by guiding the pricing step and reducing unnecessary iterations.

The Cutting Plane method also benefited from the enhanced framework, showing noticeable reductions in computation time. However, its performance remained limited by the iterative generation of constraints, making it less efficient than Column Generation for larger instances.

Benders Decomposition showed improvement with the enhanced framework, particularly in reducing expensive feasibility checks. However, due to the overhead of solving master and subproblems iteratively, it remained less scalable. The baseline version became computationally infeasible even for moderate problem sizes, while the enhanced version handled larger instances but still lagged behind Column Generation.

Overall, all methods produced consistent and correct solutions, but their computational efficiency differed significantly. Column Generation emerged as the most effective approach for this problem.

4.7 Discussion

The results of the flight scheduling problem reinforce the observations made in the hospital setting while also highlighting problem-specific differences.

One key insight is that structured optimization methods combined with LLM-based reasoning significantly improve performance. The ReAct mechanism enables iterative refinement, allowing the model to detect infeasibilities early and guide the search process more effectively.

Another important observation is the role of problem structure. The flight scheduling problem includes strict constraints such as time compatibility, location continuity, and duty limits, which naturally restrict the feasible solution space. As a result, improvements from LLM integration are more controlled compared to the hospital problem, where combinatorial complexity is higher.

Column Generation benefits the most from this integration because it directly leverages dual information to guide search. The LLM enhances this process by proposing high-quality columns, leading to faster convergence.

In contrast, Cutting Plane and Benders methods still rely on iterative constraint generation and validation, which introduces overhead even when guided by LLM reasoning. While performance improves, these methods remain less efficient for large-scale instances.

These results demonstrate that LLMs are most effective when used to guide structured optimization steps rather than replace them.

4.8 Chapter Summary

In this chapter, the OPRO + ReAct framework was applied to the flight crew scheduling problem, extending the analysis beyond the hospital domain.

Three optimization methods—Column Generation, Cutting Plane, and Benders Decomposition—were evaluated. The results show that integrating structured reasoning into the optimization loop significantly improves computational efficiency while main-

taining solution correctness.

Column Generation emerged as the most effective method due to its scalability and efficient use of dual information. Cutting Plane and Benders Decomposition also benefited from the enhancement but remained less competitive for larger instances.

The findings confirm that the proposed framework generalizes well across different optimization problems and consistently improves performance. This demonstrates its potential for broader applications in real-world scheduling and resource allocation problems.

CHAPTER 5

Conclusion and Future Work

5.1 Conclusion

This thesis presented an enhanced optimization framework combining OPRO with ReAct-based reasoning for solving complex scheduling problems. The approach was evaluated on two domains: hospital scheduling and flight crew assignment.

Across both problems, the integration of structured reasoning into classical optimization methods led to significant improvements in computational efficiency, stability, and scalability. The enhanced framework consistently produced solutions of the same quality as traditional methods while reducing computation time.

Among the methods studied, Column Generation emerged as the most effective approach due to its ability to efficiently explore the solution space using dual information. The addition of LLM-guided reasoning further improved its convergence and reduced unnecessary computations.

Cutting Plane and Benders Decomposition also benefited from the framework, particularly in reducing combinatorial search. However, their reliance on iterative constraint generation and validation limited their scalability compared to Column Generation.

Overall, the results demonstrate that LLMs can effectively complement classical optimization techniques by guiding search and improving decision-making within iterative frameworks.

5.2 Future Work

Several directions can be explored to extend this work further.

First, the framework can be applied to more complex real-world scheduling problems, such as airline network planning, workforce allocation, and supply chain optimization.

Second, integrating domain-specific fine-tuning of language models may further improve the quality of generated solutions and reduce reliance on validation steps.

Third, hybrid approaches combining reinforcement learning with optimization and LLM reasoning could provide adaptive strategies for dynamic environments.

Finally, improving the theoretical understanding of LLM-guided optimization, including convergence guarantees and robustness, remains an important area for future research.

5.3 Final Remarks

This work highlights the potential of combining artificial intelligence with traditional optimization methods. Rather than replacing classical techniques, LLMs act as intelligent guides that enhance performance and scalability.

As optimization problems continue to grow in complexity, such hybrid approaches are likely to play a crucial role in advancing both research and practical applications.

References

Artigues, Christian, Philippe Michelon, and Stéphane Reusser (2003). "A serial insertion schedule generation scheme for resource-constrained project scheduling". In: *Computers & Operations Research* 30.3, pp. 395–413. URL: https://www.researchgate.net/publication/220405305_A_Serial_Insertion_Schedule_Generation_Scheme_for_Resource-constrained_Project_Scheduling.

Bellman, Richard (1957). *Dynamic Programming*. Reprinted by Dover Publications in 2003. Princeton, NJ: Princeton University Press.

Bùi, Hòa T. and Alberto De Marchi (2025). *Cutting plane methods with gradient-based heuristics*. arXiv: 2511.00520 [math.OC]. URL: https://arxiv.org/abs/2511.00520.

Chambolle, Antonin (2022). *Continuous optimization, an introduction*. URL: https://www.ceremade.dauphine.fr/~chambolle/data/medias/optimisation/coursopti2024.pdf.

Ciré, Andre A., Elvin Çoban, and John N. Hooker (2016). "Logic-based Benders decomposition for planning and scheduling: a computational analysis". In: *The Knowledge Engineering Review* 31.5, pp. 440–451. DOI: 10.1017/s0269888916000254.

Land, Ailsa H. and Alison G. Doig (1960). "An Automatic Method of Solving Discrete Programming Problems". In: *Econometrica* 28.3, pp. 497–520. DOI: 10.2307/1910129.

Nemhauser, George L. and H. P. Williams (1990). "Integer and Combinatorial Optimization". In: *The Journal of the Operational Research Society* 41.12, pp. 1181–1182. DOI: 10.2307/2583737.

Pal, Soumitra (2005). *Column Generation*. M.Tech Seminar Report. Department of Computer Science and Engineering. Indian Institute of Technology, Bombay. URL: https://www.cse.iitb.ac.in/~mitra/mtp/seminar/report/colgen.pdf.

Yang, Chengrun et al. (2023). “Large Language Models as Optimizers”. In: *arXiv preprint arXiv:2309.03409*. DOI: 10.48550/arXiv.2309.03409.

Yao, Zifan, Yutong Hao, and Jingyu Zhang (2023). “Overview of Greedy Algorithms and Applications”. In: *Highlights in Science, Engineering and Technology* 38, pp. 262–267. URL: https://www.researchgate.net/publication/376076453_Review_on_greedy_algorithm#full-text.